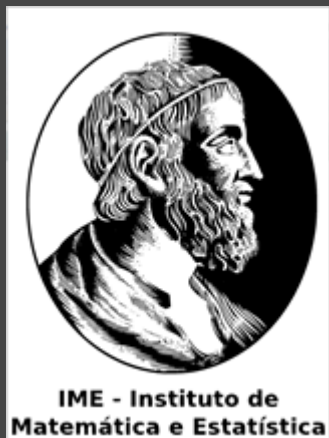




# Evolução de Software

## Ferramentas, técnicas e métricas

[versão 1.1]

Gustavo Oliva, Mauricio Aniche, Marco Gerosa  
{goliva, aniche, gerosa}@ime.usp.br

# Instrutores

2

## □ Gustavo Oliva

- Mestre em Ciência da Computação pelo IME/USP
  - Evolução e manutenção de software
  - Gerência de dependências em sistemas OO
- Atuou como desenvolvedor na IBM Brasil por ~3 anos

## □ Mauricio Aniche

- Mestre em Ciência da Computação pelo IME/USP
  - Test-Driven Development e Design de Sistemas OO
- Instrutor dos cursos de Java e Métodos Ágeis da Caelum

# Objetivos do Curso

3

- ❑ Objetivo Geral

- ▣ Discutir evolução de software e técnicas para extração e visualização de dados

- ❑ Objetivos Específicos

- ▣ Discutir ferramentas
  - ▣ Discutir técnicas
  - ▣ Discutir métricas

# Agenda

4

- Motivação
- Conceitos Básicos
- Temas Atuais de Pesquisa
- Métricas e Visualização
- XFlow e rEvolution

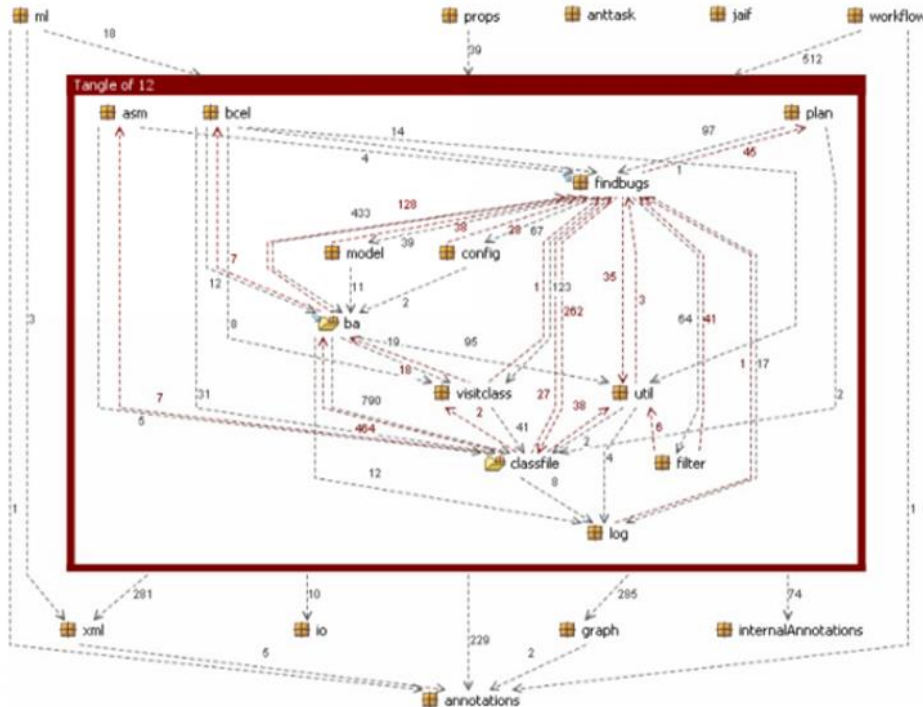
# Agenda

5

- **Motivação**
- Conceitos Básicos
- Temas Atuais de Pesquisa
- Métricas e Visualização
- XFlow e rEvolution

# O mundo real...

6



... é complicado!

# Todo software útil...

7

- ❑ Muda continuamente
- ❑ Tende a tornar-se mais complexo
- ❑ Tende a crescer



# Motivação

8

- A evolução do software é **difícil de compreender**
  - ▣ Grande quantidade de dados históricos
  - ▣ A interação entre aspectos técnicos e sociais do processo de desenvolvimento de software é difícil de desvendar
  - ▣ Uma análise abrangente da evolução requer mecanismos sofisticados, como visualizações sob várias perspectivas e cálculo de métricas

# Evolução de Software

9

- Evolução de software se preocupa principalmente com as mudanças do sistema em relação a diferentes versões ou releases do mesmo
- Em Maio de 2010, o Google Scholar reportou que, em 2009, **70 publicações** continham “software evolution” no título, e **mais de 900** tinham “software evolution” em algum lugar do texto

# Evolução de Software

10

- ❑ Sistemas de software **precisam evoluir**
  - ▣ A sociedade altamente dinâmica impõe uma pressão constante para **mudanças** em sistemas de software
  - ▣ Para continuar útil, é crucial que sistemas de software possam ser facilmente adaptáveis a mudanças contínuas e flexíveis o suficiente para adição de novas funcionalidades
  - ▣ Software que não é tolerante a modificações está fadado ao **abandono** ou a **substituição** [Mens & Demeyer 2008], [Bode 2009]

# Estudos em Evolução de Software

11

- Estudos sobre evolução possuem **diversos objetivos**
  - ▣ Identificação de boas práticas de engenharia de software baseadas na análise de como sistemas de sucesso e de longo prazo se mantêm [Bennet & Rajlich 2000]
  - ▣ Avaliação de hipóteses empíricas sobre características da evolução de software [Lehman 1997]
  - ▣ Padrões de colaboração entre desenvolvedores [Cataldo 2008]
  - ▣ Identificação de problemas de design e de código através da visualização do histórico do software [D'Ambros & Lanza 2010]

# Agenda

12

- Motivação
- **Conceitos Básicos**
- Temas Atuais de Pesquisa
- Métricas e Visualização
- XFlow and rEvolution

# Software Intelligence

13

## Software Intelligence: The Future of Mining Software Engineering Data

Ahmed E. Hassan  
School of Computing  
Queen's University  
Kingston, ON, Canada  
ahmed@cs.queensu.ca

Tao Xie  
Department of Computer Science  
North Carolina State University  
Raleigh, NC, USA  
xie@csc.ncsu.edu

### ABSTRACT

Mining software engineering data has emerged as a successful research direction over the past decade. In this position paper, we advocate Software Intelligence (SI) as the future of mining software engineering data, within modern software engineering research, practice, and education. We coin the name SI as an inspiration from the Business Intelligence (BI) field, which offers concepts and techniques to improve business decision making by using fact-based support systems. Similarly, SI offers software practitioners (not just developers) up-to-date and pertinent information to support their daily decision-making processes. SI should support decision-making processes throughout the lifetime of a software system not just during its development phase.

The vision of SI has yet to become a reality that would enable software engineering research to have a strong impact on modern software practice. Nevertheless, recent advances in the Mining Software Repositories (MSR) field show great promise and provide strong support for realizing SI in the near future. This position paper summarizes the state of practice and research of SI, and lays out future research directions for mining software engineering data to enable SI.

### Categories and Subject Descriptors

D.2.7 [Software Engineering]: Distribution, Maintenance, and Enhancement

### General Terms

Documentation, Economics, Experimentation, Human Factors, Management, Measurement, Reliability, Verification

### Keywords

Software intelligence, mining software engineering data, mining software repositories

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.  
FOSE 2010, November 7-8, 2010, Santa Fe, New Mexico, USA.  
Copyright 2010 ACM 978-1-4503-0427-6/10/11...\$5.00

### 1. INTRODUCTION

Much of software practice centers around daily decisions and questions (e.g., when to release a software system? which parts of a software system to change? which parts of a software system to test? who is using this feature? and who knows about this feature?). Unfortunately, nowadays many decisions related to a software system are based on intuition and gut feeling. Determining when a software system is ready for release, whether a part of a software system should be re-factored or re-written, or which parts of a software system should be thoroughly tested is a matter of art instead of a well-studied science. These haphazard decision processes lead to wasted resources and increased cost of building and maintaining large complex software systems.

Software practitioners are in dire need of what we propose to call Software Intelligence (SI). While Business Intelligence (BI) [27] offers concepts and techniques to improve business decision making by using fact-based support systems, SI offers software practitioners (not just developers) up-to-date and pertinent information to support their daily decision-making processes. SI provides practitioners with access to specialized fact-supported views of their software system so they can answer critical questions about it. Using SI, owners, maintainers, and developers of software systems can perform long-term and short-term informed strategic planning. Moreover, SI gives companies a better understanding of the true potential and actual limitation of their software assets.

Mining software engineering data has emerged as a research direction over the past decade. This research direction has already achieved substantial success in both research and practice. In this position paper, we advocate Software Intelligence (SI) as the future of mining software engineering data, within modern software engineering research, practice, and education.

The vision of SI has yet to become a reality. Nevertheless, recent advances in the Mining Software Repositories (MSR) field show great promise and provide strong support for realizing SI in the near future, as software engineering research aims to ensure its relevance and impact on modern software practice. This position paper summarizes state of practice and research of SI, and lays out future research directions of mining software engineering data to enable SI.

### 2. STATE OF PRACTICE

Prior experiences and dominant patterns are the driving force for many decision-making processes in modern software organizations. Software practitioners often rely on their experience, intuition, and gut feeling in making important decisions. Managers allocate development and testing resources based on their experience in previous projects and their intuition about the complexity of the new project relative to prior projects. Developers commonly

As mesmas ideias de  
Business Intelligence (BI), só  
que aplicadas no domínio  
de software

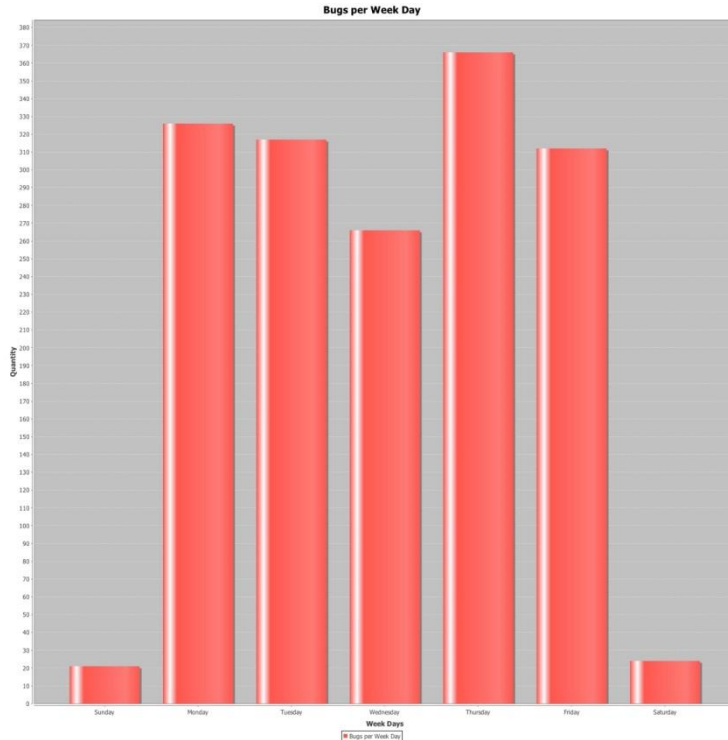
# Mineração de Repositórios de Código

14

- Mineração de Repositórios de Código (MSR) se refere à aplicação de técnicas de **mineração de dados em dados históricos de projetos** [Ball *et al.* 1996]
- Estudos sobre evolução de software frequentemente estão associados à MSR

# Dados interessantes

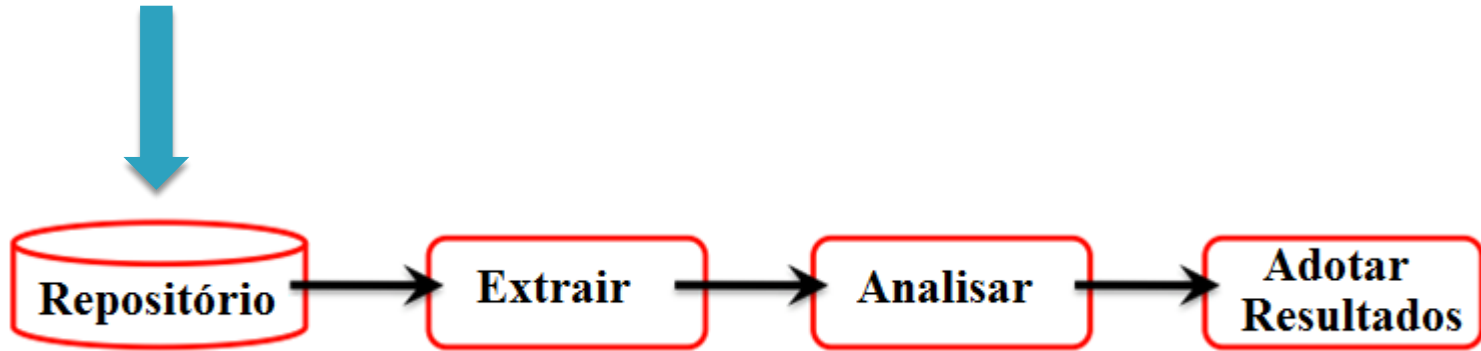
15



- As técnicas de MSR podem encontrar padrões interessantes em projetos de software
- No exemplo ao lado, o gráfico mostra o número de bugs escritos por dia da semana. Para esse projeto, podemos ver que quarta-feira é o dia em que essa equipe produz menos bugs

# Repositórios

16



# Repositórios de código

17

- Há vários deles: CVS, SVN, Git, Mercurial, Bazaar
- Muitas características em comum
- Mas muitas características específicas também!

# Benefícios

18

- ❑ Contém todo o histórico do projeto ao longo do tempo
- ❑ Contém informações relevantes, como
  - ▣ Data/hora
  - ▣ Desenvolvedor
  - ▣ Mensagem explicando a alteração (commit message)

# Dificuldades

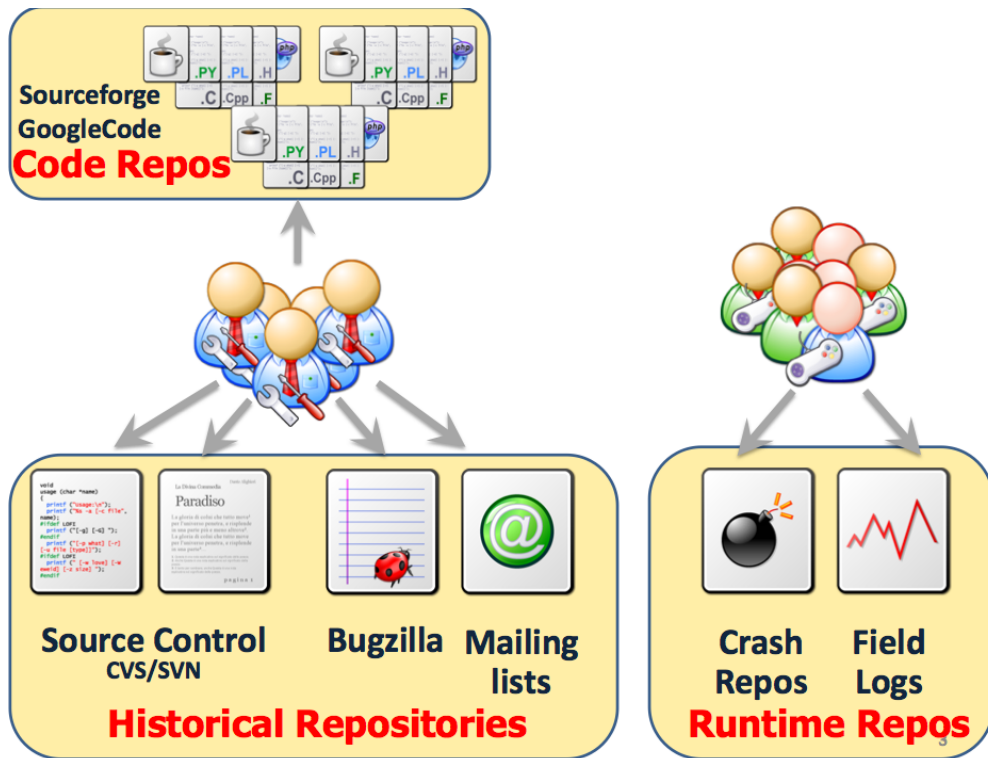
19

- ❑ Muitas equipes não colocam mensagem nos commits
- ❑ Fazem commits grandes, dificultando a compreensão do mesmo
- ❑ Dependendo do controlador de versão, extrair dados pode ser trabalhoso
  - ▣ CVS, por exemplo, não dá suporte para commits atômicos

# Outros repositórios

20

- ❑ Listas de Discussão
- ❑ Bug Trackers
- ❑ ...



# Issue Trackers

21



Commons Collections / COLLECTIONS-373

## Bug in class#ListOrderedSet with reproducible JUnit test

Log In

Views ▾

### Details

Type: Bug  
Priority: Major  
Affects Version/s: 3.2  
Component/s: None  
Labels: [patch](#)  
Environment: jdk 1.6.x

Status: Open  
Resolution: Unresolved  
Fix Version/s: None

### People

Assignee: Unassigned  
Reporter: [Sai Zhang](#)  
 Vote (0) Watch (0)

### Dates

Created: 03/Apr/11 07:43  
Updated: 18/Aug/11 01:19

### Description

Hi all:

I am writing an automated bug finding tool, and using Apache Commons Collections as an experimental subject for evaluation.

The tool creates executable JUnit tests as well as explanatory code comments. I attached one bug-revealing test as follows. Could you please kindly check it, to see if it is a real bug or not?

Also, it would be tremendous helpful if you could give some feedback and suggestion on the generated code comments?

# Issue Trackers

22

- ❑ Descrição do bug, severidade
- ❑ Data de criação do ticket, data de fechamento do ticket
- ❑ Desenvolvedor que fechou, desenvolvedores que interagiram na discussão
- ❑ ...

# Benefícios

23

- Uma base de dados grande sobre bugs e suas características
- As informações de data possibilitam a criação de *links* entre o repositório de código e o repositório de bugs

# Dificuldades

24

- ❑ Cada ferramenta armazena/exporta dados de uma maneira diferente
- ❑ Muitas equipes não usam
- ❑ O *link* entre repositório de código e repositório de bugs é fraco
  - ▣ Uma solução comum é procurar por referências a bugs nas mensagens dos commits

# Listas de Discussão

25

## JSON - Serializacao do validator.errors

VRaptor | X

☆ **Tiago Oliveira** tiagojco@gmail.com to caelum-vraptor

[show details](#) 12:24 PM (22 hours ago)

Caros,  
tenho em um projeto um formulario de fale conosco cuja submit eh ajax,  
segue abaixo trecho de código relevante:

```
//Ajax do Controller
@Post
@TransactionalScoped
public void faleConosco(FaleConosco f) {
    if (f == null)
        result.use(Results.json()).from(new Success(false)).serialize();

    validator.validate(f);
    validator.onErrorUse(Results.json()).from(new
Success(false,errors())).include("errors").serialize();

    contatoService.registrarFaleConosco(f);
    result.use(Results.json()).from(new Success(true)).serialize();
}

public static class Success {
    final boolean success;
    final Object errors;
```

# Listas de Discussão

26

- ❑ Local onde acontece uma grande troca de informações relativas ao projeto
- ❑ Discute-se bugs, melhorias, novas funcionalidades, etc.
- ❑ Geralmente disponível na web
- ❑ Bastante usado por projetos open-source

# Benefícios

27

- ❑ “Documentação” das discussões podem conter dados interessantes.
- ❑ Altamente relacionados com atividades de mudança de código [Bird *et al.* 06]
- ❑ Geralmente contém discussões sobre planos futuros, decisões de design
- ❑ Contém a “rede social” de desenvolvedores do projeto

# Exemplo: Motivação

28

| Dimension          | 1.3   | 2.0   |
|--------------------|-------|-------|
| Optimism           | -0.37 | *     |
| Tentative          | -1.3  | *     |
| References to Time | 1.1   | *     |
| Future tense verbs | -0.7  | *     |
| Social Processes   | *     | 0.74  |
| Inclusive          | *     | -0.64 |

**Table 4. Mean differences for Apache 1.3 and 2.0 releases. (\*  $p > 0.05$ , otherwise  $p \leq 0.05$ )**

- Estudo do conteúdo das mensagens antes e depois de uma release;
- Depois do release Apache 1.3, houve uma queda no otimismo;
- Depois do release do Apache 2.0, houve um aumento na socialibilidade;
- [Rigby & Hassan 07]

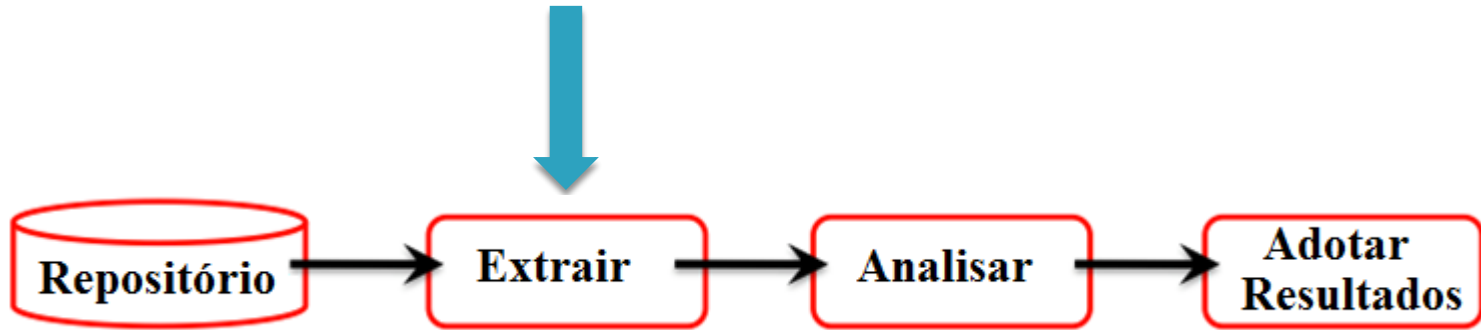
# Dificuldades

29

- ❑ Dados não estruturados dificultam a extração de dados
- ❑ Desenvolvedor com múltiplos e-mails
- ❑ Threads “quebradas” em vários e-mails diferentes

# Extração de dados

30



# Logs

31

- ❑ Controladores de versão proveem logs das modificações armazenadas
- ❑ Geralmente em texto puro
- ❑ Exemplo: *git log*

```
commit 2444af13bd6439dd7ae1293c3f54482bb499ee74
Author: Mauricio Aniche <mauricioaniche@gmail.com>
Date:   Tue Aug 30 11:07:22 2011 -0300
```

menos conexoes no c3p0 pra aguentar mais caras

```
commit 1c3ad7cfbfd80747f94fd9d6e955e1db2e8bdbef
Author: Mauricio Aniche <mauricioaniche@gmail.com>
Date:   Mon Aug 29 12:10:13 2011 -0300
```

melhorando cara da agenda

```
commit dea1d3e7d9a47ed8edcc0d81cabd25ccc20ea99c
Author: Mauricio Aniche <mauricioaniche@gmail.com>
Date:   Sun Aug 28 13:03:35 2011 -0300
```

mexendo nos relatorios de novo

```
commit fc28f97b24e90d41547264ae58ef7062529ac22e
Author: Mauricio Aniche <mauricioaniche@gmail.com>
Date:   Sun Aug 28 12:31:54 2011 -0300
```

acertando url das imagens

# Exemplo do diff

32

- Diffs mostram a diferença entre uma versão e a versão anterior
- Útil para identificar as mudanças feitas

```
commit 569309ceb973fdd286fb6fdbec31dfcd9faa3db
Author: Mauricio Aniche <mauricioaniche@gmail.com>
Date:   Wed Jul 13 22:51:08 2011 -0300
```

exportando excel

```
diff --git a/src/integration-test/java/br/com/easyclinica/infra/dao/PrecifiedThingsDaoTests.java b/src/integration-test/java/br/com/easyclinica/infra/dao/PrecifiedThingsDaoTests.java
index 01ae603..f0c4ff2 100644
--- a/src/integration-test/java/br/com/easyclinica/infra/dao/PrecifiedThingsDaoTests.java
+++ b/src/integration-test/java/br/com/easyclinica/infra/dao/PrecifiedThingsDaoTests.java
@@ -20,6 +20,7 @@ import br.com.easyclinica.domain.entities.PrecifiedProcedure;
 import br.com.easyclinica.domain.entities.PrecifiedSpecialty;
 import br.com.easyclinica.domain.entities.Procedure;
 import br.com.easyclinica.domain.entities.Specialty;
+import br.com.easyclinica.domain.entities.pricing.PricedProcedure;
 import br.com.easyclinica.domain.entities.pricing.PricedStuff;
 import br.com.easyclinica.tests.helpers.HealthCarePlanBuilder;
 import br.com.easyclinica.tests.helpers.MaterialBuilder;
@@ -233,11 +234,13 @@ public class PrecifiedThingsDaoTests extends DaoBase {
     precifiedProcedure.setProcedure(procedure1);
     session.save(precifiedProcedure);

-    List<PricedStuff> procedures = dao.getProceduresPrice(healthCarePlan);
+    List<PricedProcedure> procedures = dao.getProceduresPrice(healthCarePlan);

     assertEquals(2, procedures.size());
     assertEquals(precifiedProcedure.getFixedAmount().doubleValue(), procedures.get(0).getAmount().doubleValue(), 0);
     assertEquals(precifiedProcedure.getRoomTaxAmount().doubleValue(), procedures.get(0).getRoomTaxAmount().doubleValue(), 0);
+    assertEquals(precifiedProcedure.getCh(), procedures.get(0).getCh());
+    assertEquals(procedure1.getAmbCode(), procedures.get(0).getAmbCode());
+    assertEquals(procedure1.getName(), procedures.get(0).getName());
+    assertEquals(procedure1.getId(), procedures.get(0).getId());
+    assertEquals(BigDecimal.ZERO, procedures.get(1).getAmount());
diff --git a/src/main/java/br/com/easyclinica/domain/entities/pricing/PricedProcedure.java b/src/main/java/br/com/easyclinica/domain/entities/pricing/PricedProcedure.java
new file mode 100644
index 0000000..0144267
```

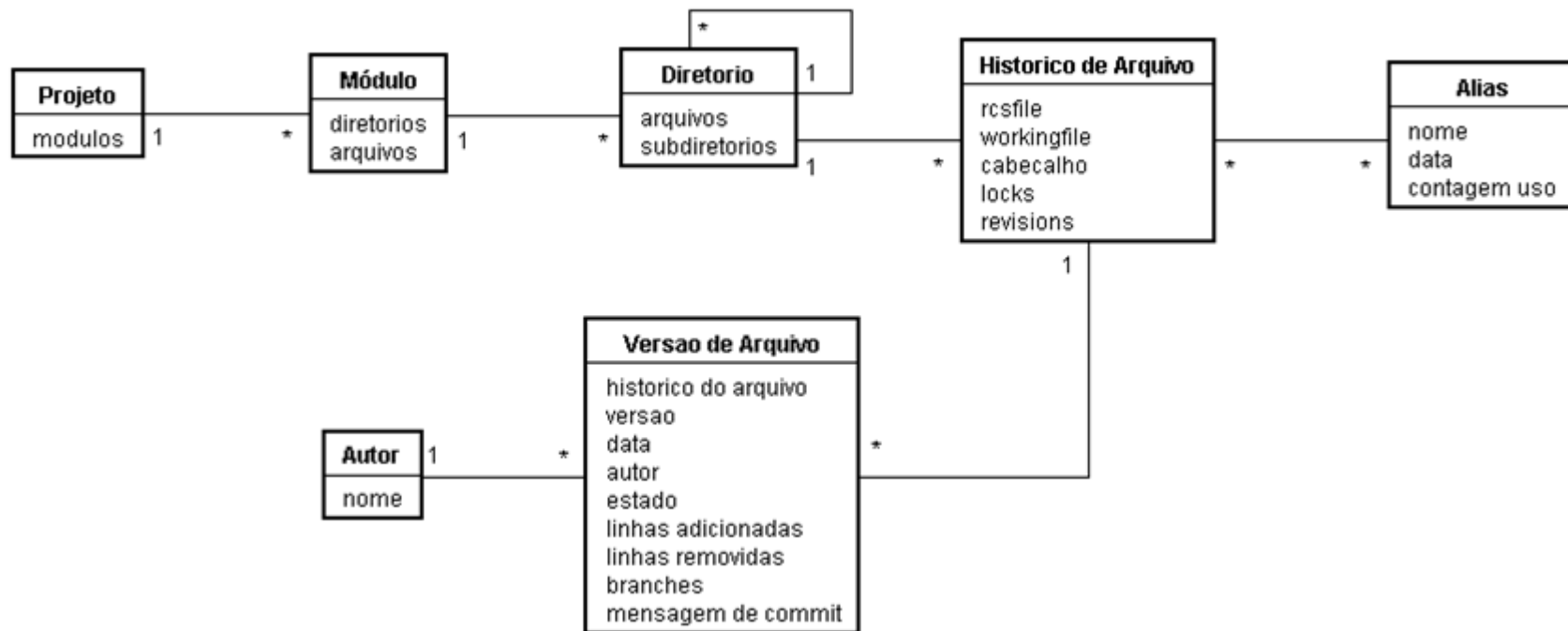
# Ferramentas prontas

33

- ❑ SVNKit ([svnkit.com](http://svnkit.com))
- ❑ JGit ([eclipse.org/jgit/](http://eclipse.org/jgit/))

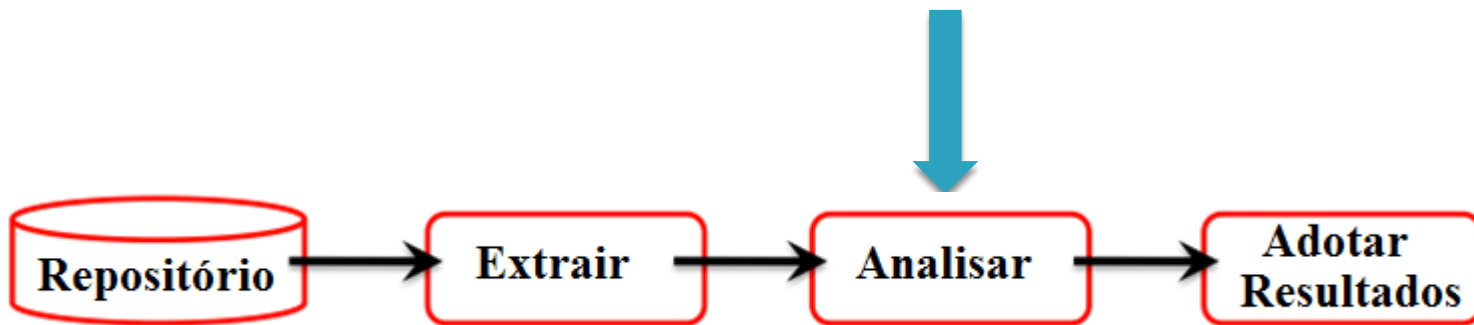
# Modelando os dados

34



# Analizando os dados

35



# Dependências Lógicas

36

- Dependências lógicas são dependências implícitas entre artefatos de software **que evoluíram juntos** [Gall *et al.* 1998]
- Dependências lógicas ocorrem **quando dois artefatos são frequentemente mudados juntos**
- Esta técnica pode revelar links evolucionários entre quaisquer tipos de artefatos que compõem o sistema, incluindo arquivos de configuração e documentação
- Dependências lógicas são **identificadas** por meio de **análise dos logs** dos sistemas de controle de versão

# Requisitos de Coordenação

37

- Requisitos de coordenação (CRs) são empregados para inferir **dependências entre desenvolvedores** [Cataldo *et al.* 2006]
- CRs são inferidas a partir de dois elementos
  - ▣ Dependências lógicas entre arquivos e
  - ▣ Alocação de tarefas para desenvolvedores
- Exemplo #1: desenvolvedores mudando o mesmo conjunto de arquivos
- Exemplo #2: desenvolvedor d1 deve coordenar seu trabalho com os desenvolvedores d2 e d3 quando os arquivos modificados por d1 dependem logicamente de arquivos modificados por d2 e d3

# Predição de Bugs

38

- A mensagem do commit pode ajudar a identificar “commits bugados” [Eyolfson, Tan, Lam 11];
  - ▣ Ao olhar a mensagem, podemos encontrar mensagens do tipo “bug corrigido”
  - ▣ Basta olhar para trás, e ver quais commits introduziram as linhas que foram removidas no commit corrente
  - ▣ Esses commits podem ser considerados “bugados”
- Possibilita a visualização dos dias e horários em que os bugs são gerados

# Predição de Mudanças

39

- Com base em métricas aplicadas sobre dependências lógicas, é possível fazer previsões sobre mudanças
  - ▣ Ideia básica: “Clientes que compraram X, também compraram Y”
  - ▣ A granularidade pode variar: classes, métodos, etc.
- É possível ainda prevenir erros decorrentes de mudanças incompletas

# Clones de código

40

- Com o histórico da base de código em mãos, é possível procurar por clones de código
  - ▣ É possível, inclusive, descobrir quando os clones foram criados

# Complexidade de código

41

- O cálculo da complexidade do código (ou qualquer outra métrica) ao longo do tempo pode passar informações interessantes como:
  - ▣ Classes mais complexas
  - ▣ Classes cuja complexidade tem aumentado constantemente

# Truck Factor

42

- ❑ A análise de desenvolvedores e artefatos modificados podem passar informações interessantes, como:
  - ▣ Artefatos que só são modificados por um desenvolvedor ao longo do tempo (truck factor)
  - ▣ Artefatos que são modificados por vários desenvolvedores e a relação disso com a complexidade do código gerado

# Ferramentas de métricas

43

□ Existem várias, como:

- JDepend/NDepend
- JavaNCSS
- Eclipse Metrics
- Kalibro Metrics
- Byecycle
- iPlasma



*Neste caso, o desenvolvedor fica responsável por analisar as várias releases (ou versões) do código para chegar a conclusões acerca da evolução do sistema em questão*

# JDepend/NDepend

44

- ❑ Calcula métricas, tais como
  - ▣ Número de classes e interfaces
  - ▣ Acoplamento aferente/eferente
  - ▣ Abstração
  - ▣ Instabilidade
  - ▣ Distância da linha principal
  - ▣ Ciclos de dependência entre pacotes
- ❑ Precisa do código compilado para executar

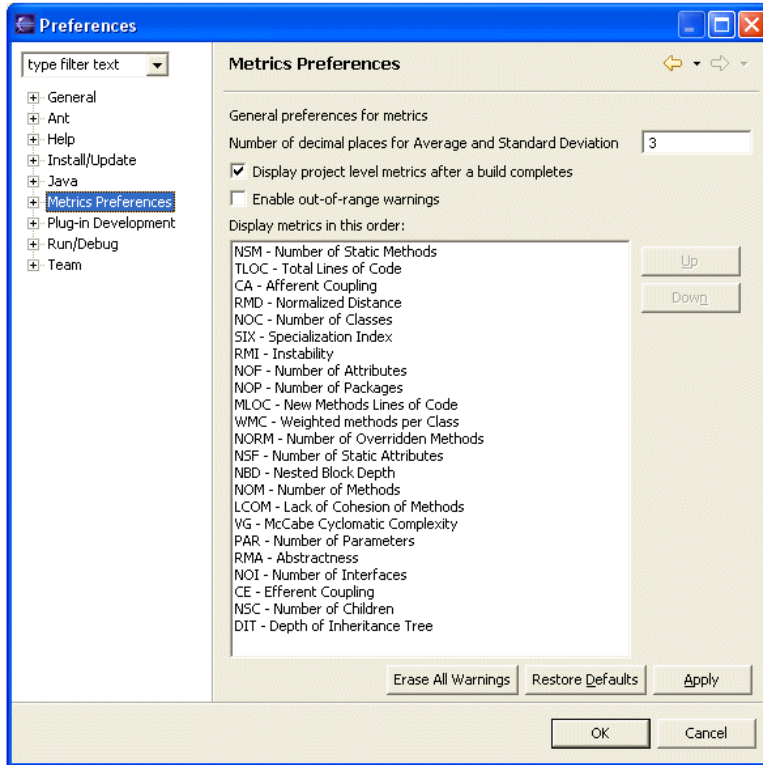
# JavaNCSS

45

- ❑ Cálcula complexidade ciclomática de métodos e classes
- ❑ Faz a análise em cima do próprio código-fonte, não exigindo o código compilado

# Eclipse Metrics

46



- ❑ Plugin do eclipse que constantemente calcula as métricas
- ❑ Uma grande quantidade de métricas diferentes
- ❑ Pode rodar fora do Eclipse
  - ▣ É complicado, mas possível

- ❑ Software brasileiro
- ❑ Fácil de plugar novas métricas
- ❑ Já possui muitas métricas implementadas
- ❑ Não precisa de código compilado

# Byecycle

48

- ❑ Mostra as dependências entre os diversos pacotes do projeto
- ❑ Plugin do Eclipse

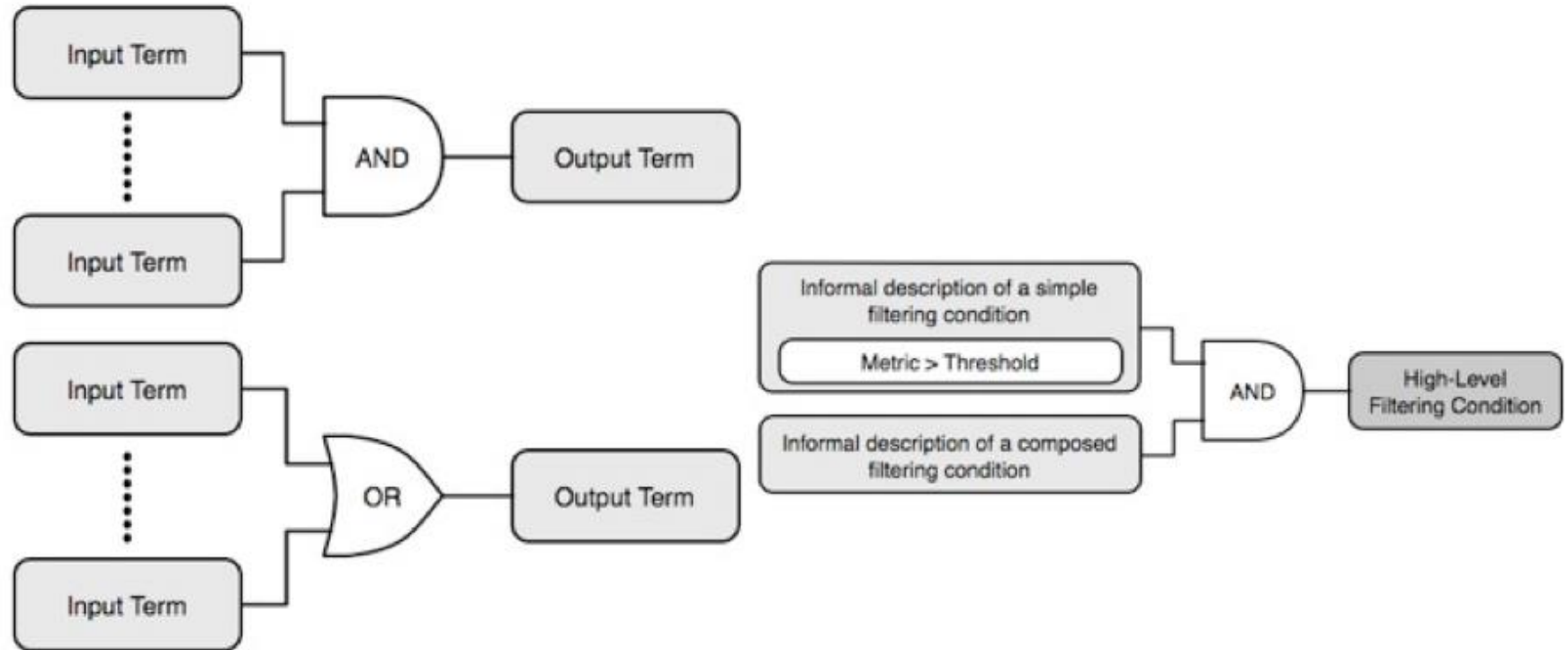
# iPlasma - Avaliando Design

49

- Além de investigar dependências, há outros métodos mais automatizados
- Lanza e Marinescu propõem um conjunto de estratégias de detecção de problemas de design com base em composição de métricas e limiares

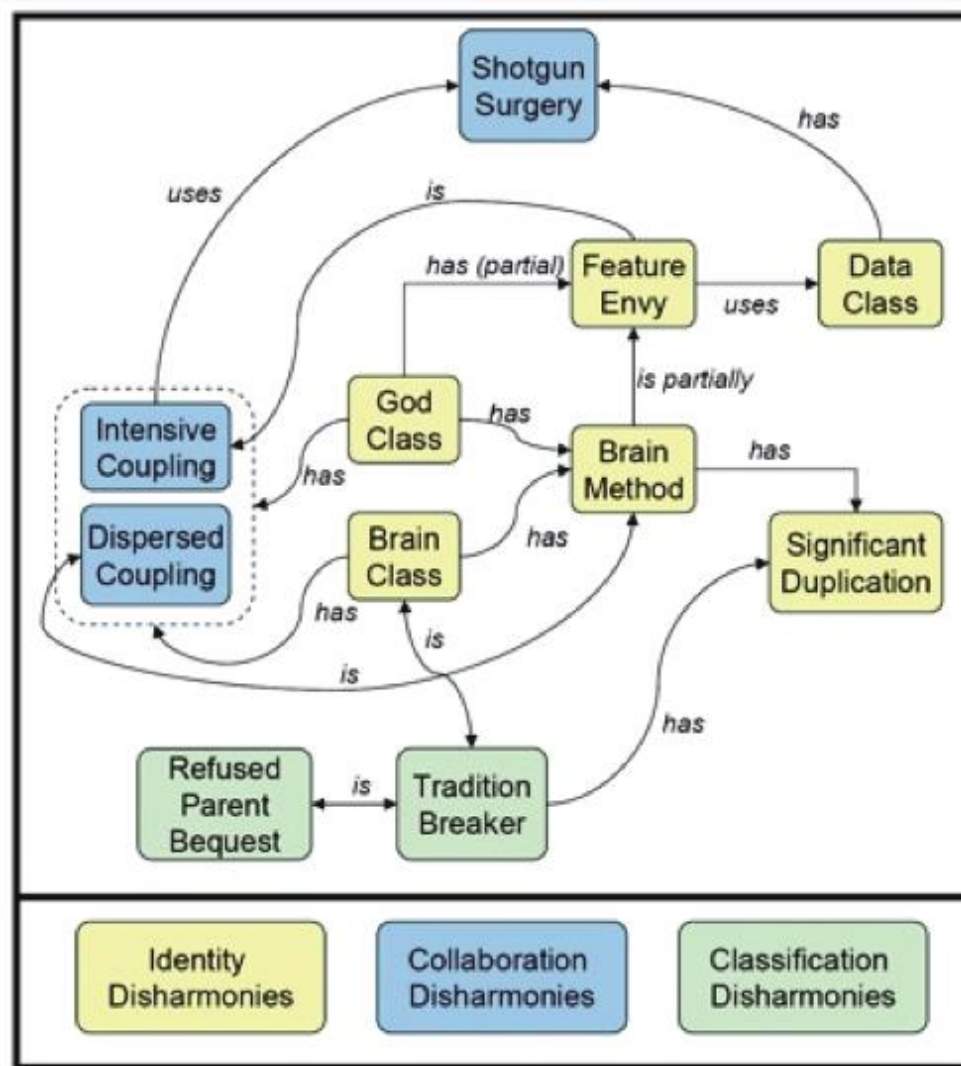
# Estratégias de Detecção

50



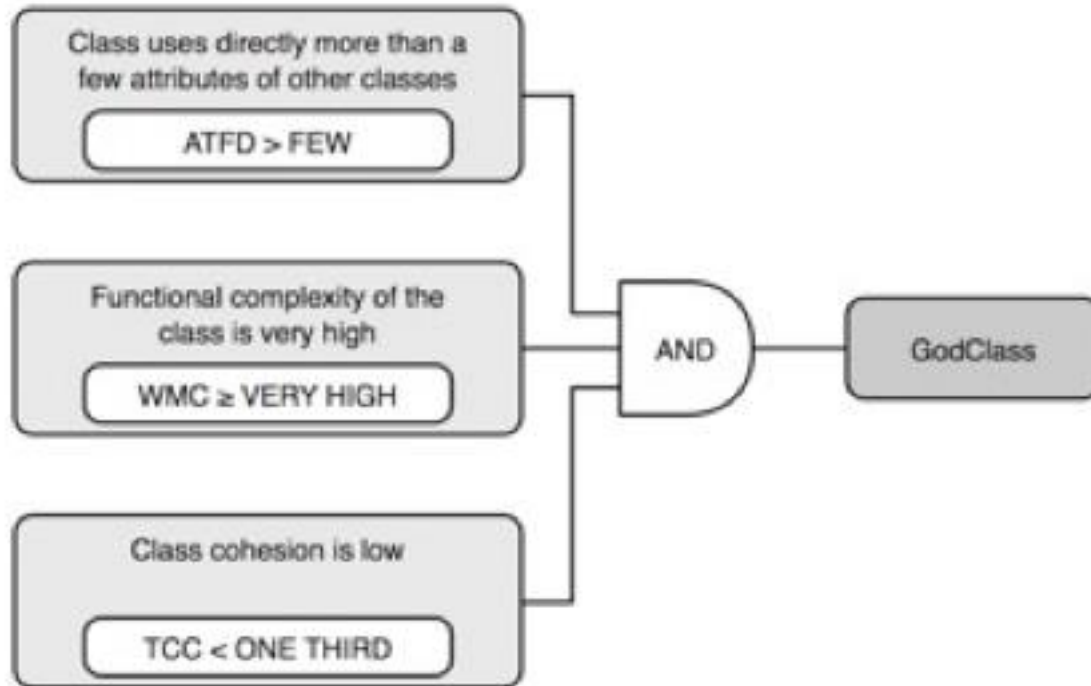
# Design Disharmonies

(Lanza e Marinescu – Object-Oriented Metrics in Practice)



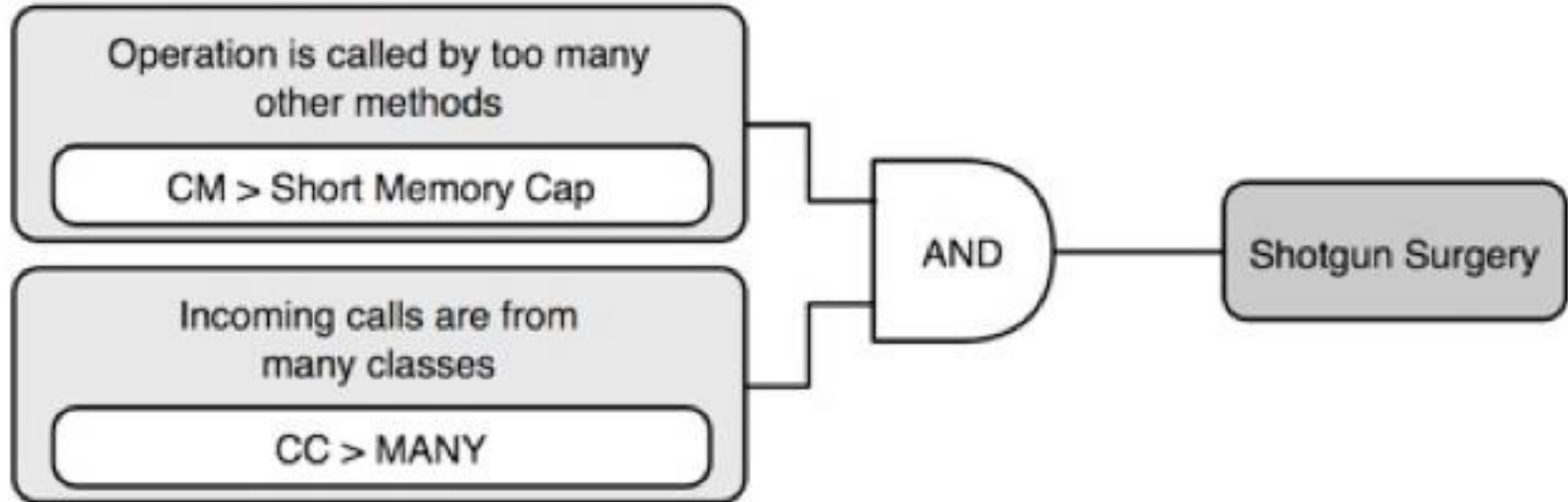
# God Class

52



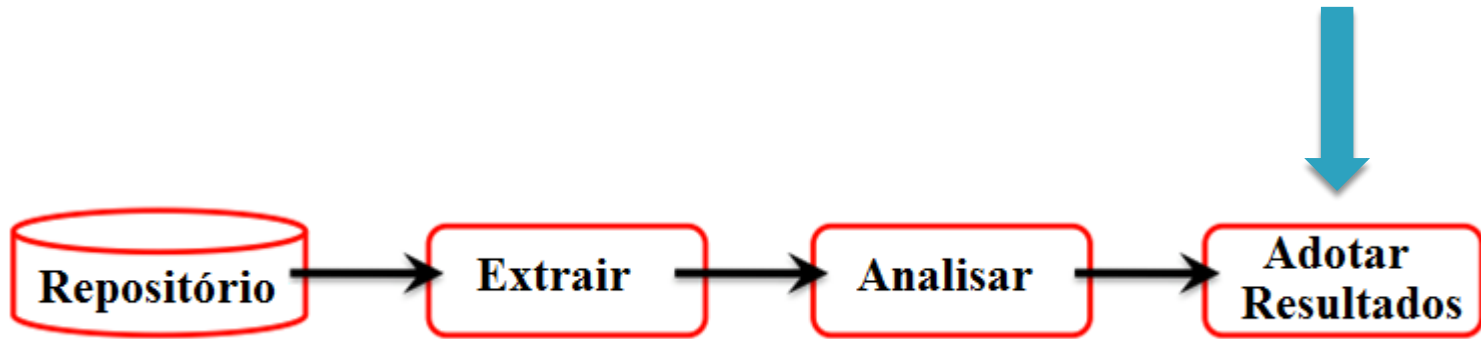
# Shotgun Surgery

53



# Adotando resultados

54



# Reengenharia

55

- Após a análise dos dados, os problemas identificados são tratados por meio de técnicas de reengenharia
  - ▣ Reengenharia (...) é o exame e alteração de um sistema para reconstituí-lo em uma nova forma e a subsequente implementação da nova forma [Chikofsky 1990]
- No escopo de sistemas orientados a objetos, há um enorme corpo de conhecimento acerca de técnicas de reengenharia
  - ▣ Padrões de reengenharia
    - Object-Oriented Reengineering Patterns [Demeyer et al. 2009]
  - ▣ Refatoração
    - Refactoring to Patterns [Kerievsky 2004]
    - Refactoring: Improving the Design of Existing Code [Fowler et al. 1999]

# Como implantar na indústria?

56

- No PASED 2011, Ahmed Hassan comentou sobre o problema do “ovo e da galinha”
  - ▣ Para mostrar o valor das técnicas de MSR, precisamos executá-las na indústria
  - ▣ Mas para que a indústria compre a ideia, é preciso primeiro mostrar o valor

# Agenda

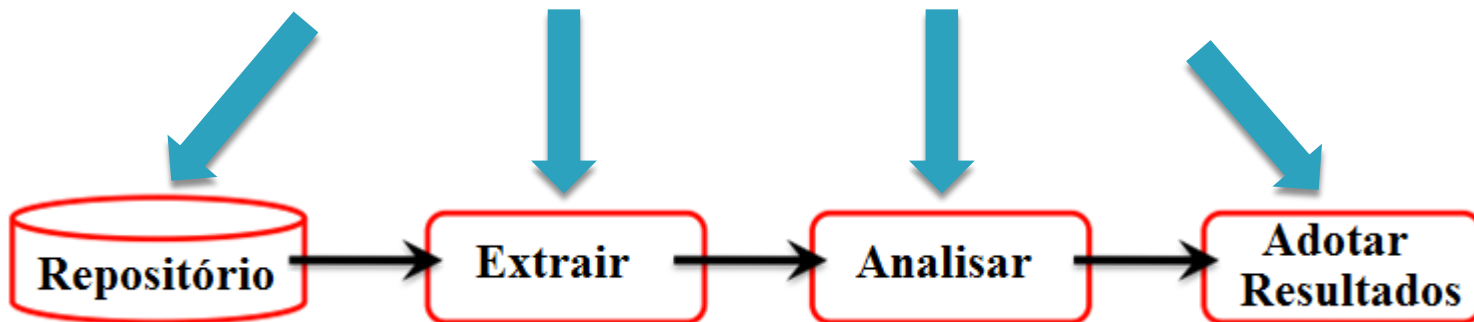
57

- Motivação
- Conceitos Básicos
- **Temas Atuais de Pesquisa**
- Métricas e Visualização
- XFlow and rEvolution

# Quais os desafios?

58

**Quais os desafios no processo de  
MSR?**



# Desafios em MSR

59

- ❑ Exploração de dados não-estruturados
  - ▣ E-mails, comentários, ...
- ❑ Relação de dados entre diferentes repositórios
  - ▣ Informações sobre bugs e repositório de código
- ❑ Procurar dados em lugares não convencionais
- ❑ Entender as limitações
  - ▣ Poucos commits, poucos desenvolvedores, ...

# Desafios em MSR

60

- ❑ Lidar com ruídos nos dados
- ❑ Melhorar a qualidade dos dados nesses repositórios
- ❑ Facilitar a extração de dados

# Desafios em MSR

61

- ❑ Entender a necessidade dos praticantes
- ❑ Mostrar os resultados práticos
- ❑ Avaliar projetos que não sejam open-source
- ❑ Simplificar o acesso a tecnologia
- ❑ Ajudar o participante a tomar decisões sobre o projeto

# Agenda

62

- Motivação
- Conceitos Básicos
- Temas Atuais de Pesquisa
- **Métricas e Visualização**
- XFlow and rEvolution

# Visualização de software

63

- A área de “Visualização de Software” estuda a **representação visual estática ou animada da informação** sobre um sistema de software baseado em sua estrutura, comportamento ou evolução [Diehl 2010]
- O **objetivo central** da visualização de software é apoiar a compreensão e análise de sistemas e algoritmos
  - ▣ Pode ser encarada como uma atividade de Engenharia Reversa

# O que é uma métrica?

64

- ❑ O mapeamento de uma característica particular de uma entidade medida para um valor numérico
- ❑ Por que medir?
  - ▣ Auxilia na administração da complexidade!

# Vantagens e Desvantagens

65

## □ Vantagens

- ▣ Capacidade de quantificar aspectos de qualidade
- ▣ Possibilidade de automatizar as medições de um sistema

## □ Desvantagens

- ▣ Números são apenas números: não confie cegamente neles!
- ▣ Em geral, capturam somente sintomas de alta granularidade , e não causas de problemas de design
- ▣ Difícil para desenvolvedores lidarem com elas
- ▣ Inflação de medições

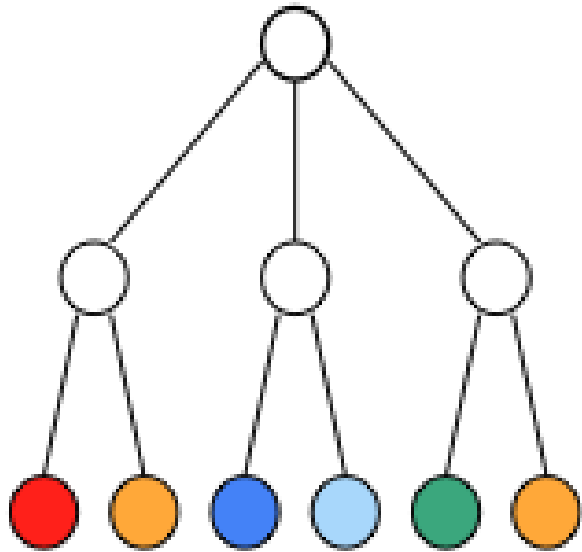
# Desafios

66

- ❑ O que visualizar?
- ❑ Como visualizar?
- ❑ Quantas coisas visualizar (ao mesmo tempo)?
- ❑ Como associar visualização e métricas?

# Um exemplo simples

67



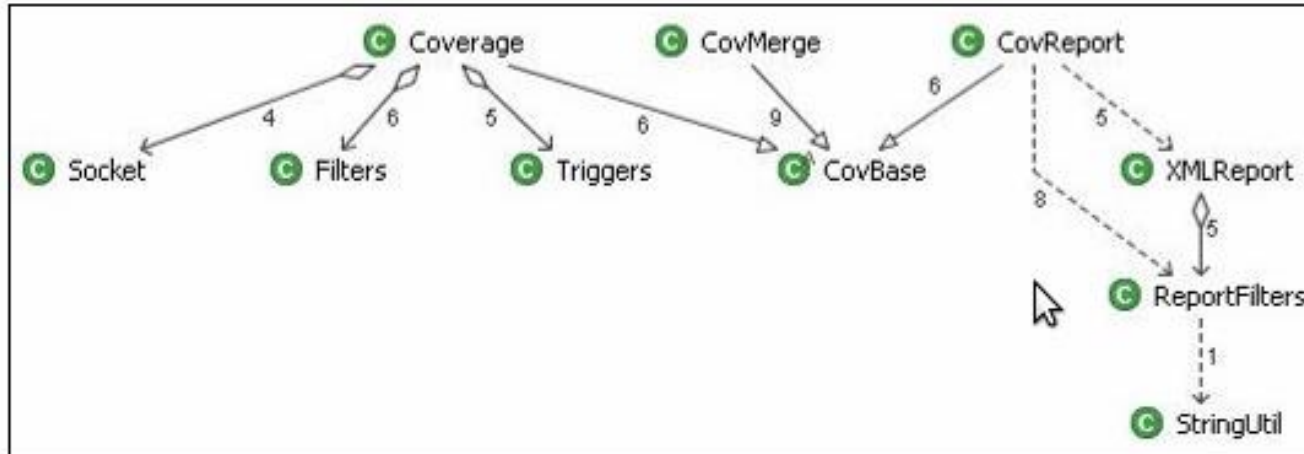
Árvore que denota uma hierarquia de classes

# Visualização de Dependências

68

## □ Structure 101 (classes)

### ▣ Grafos

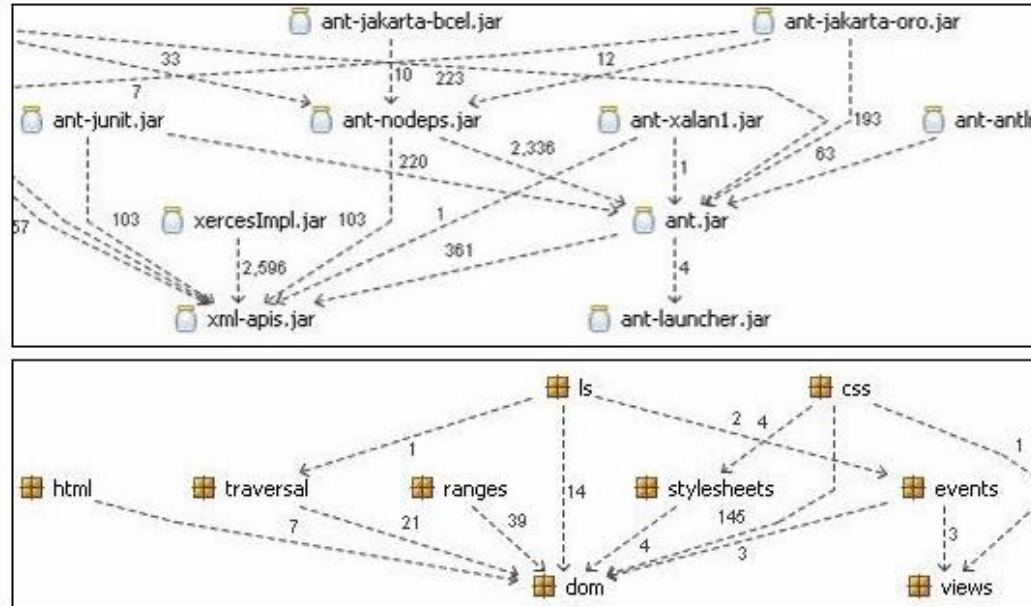


# Visualização de Dependências

69

## □ Structure 101 (pacotes e jars)

### ■ Grafos

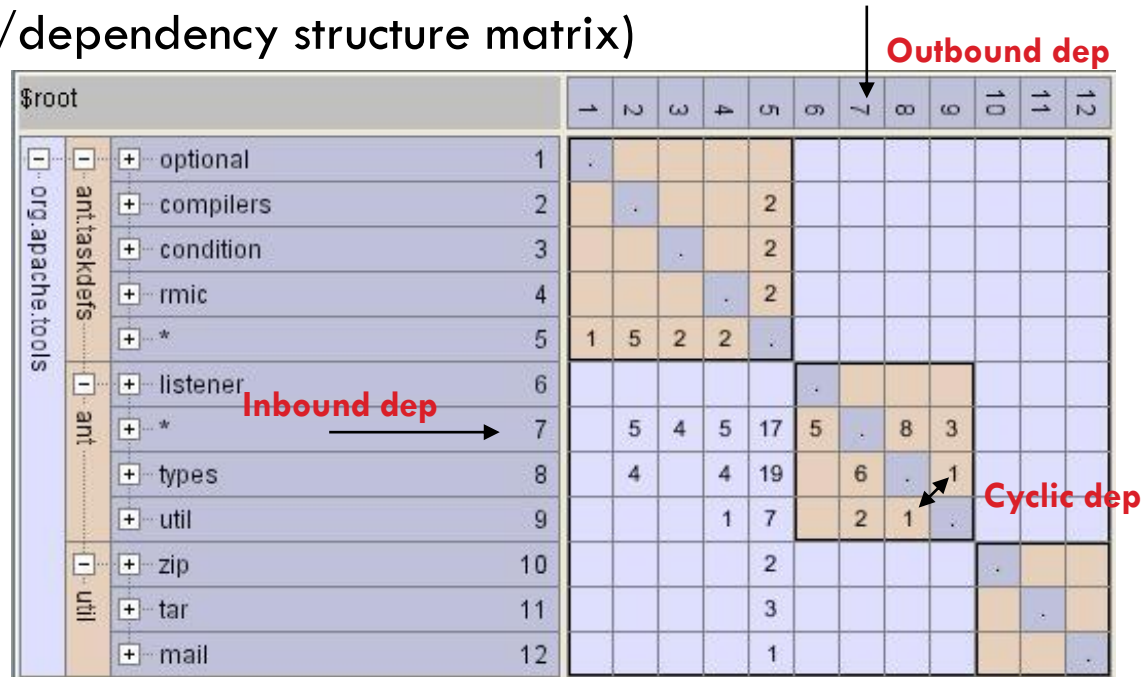


# Visualização de Dependências

70

## □ Lattix

### ▣ DSM (Design/dependency structure matrix)

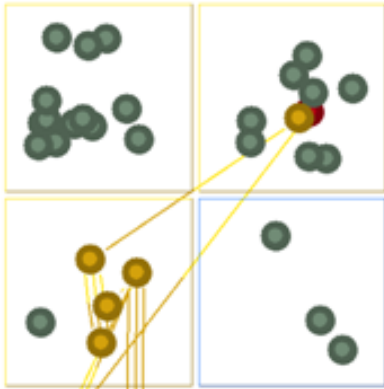


# Visualização de Dependências

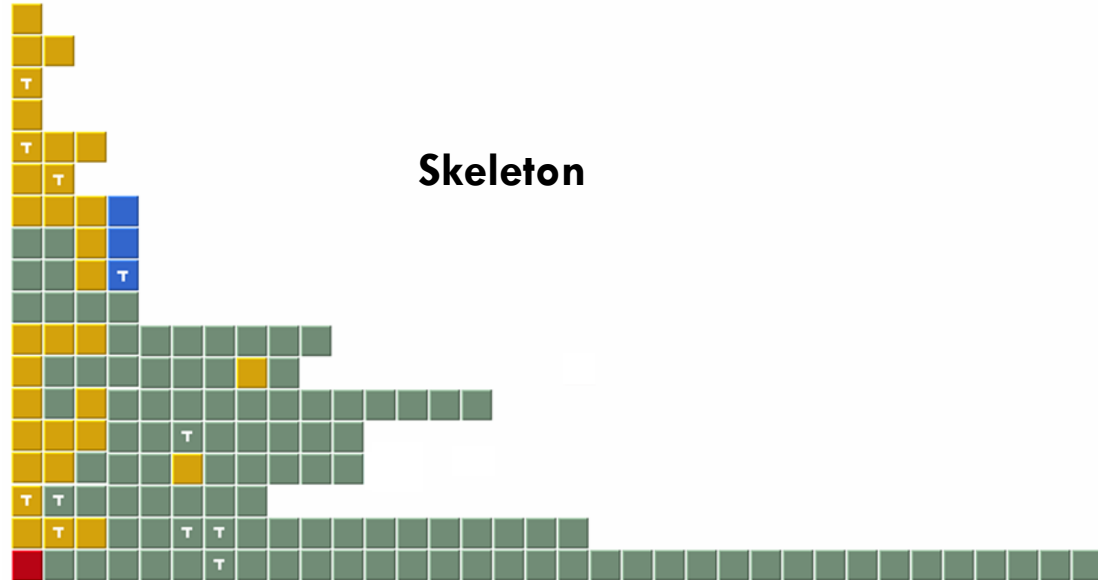
71

## □ IBM SA4J

What if



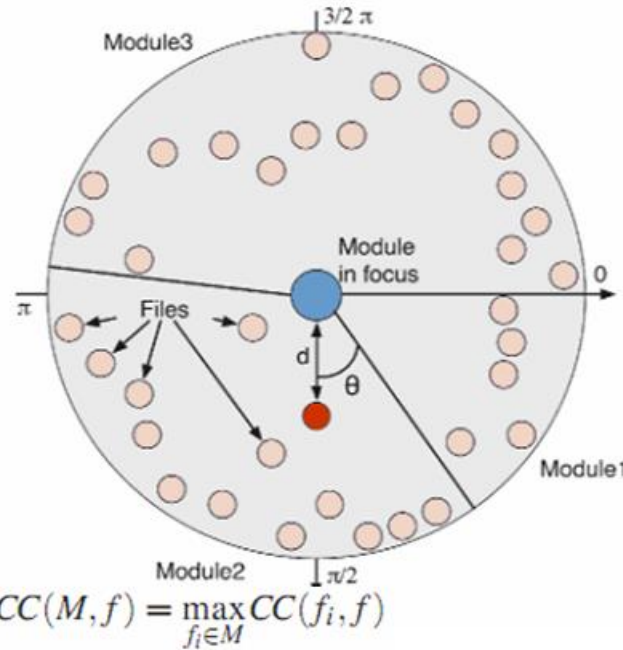
Skeleton



# Visualização de Dependências

72

## □ Evolution Radar

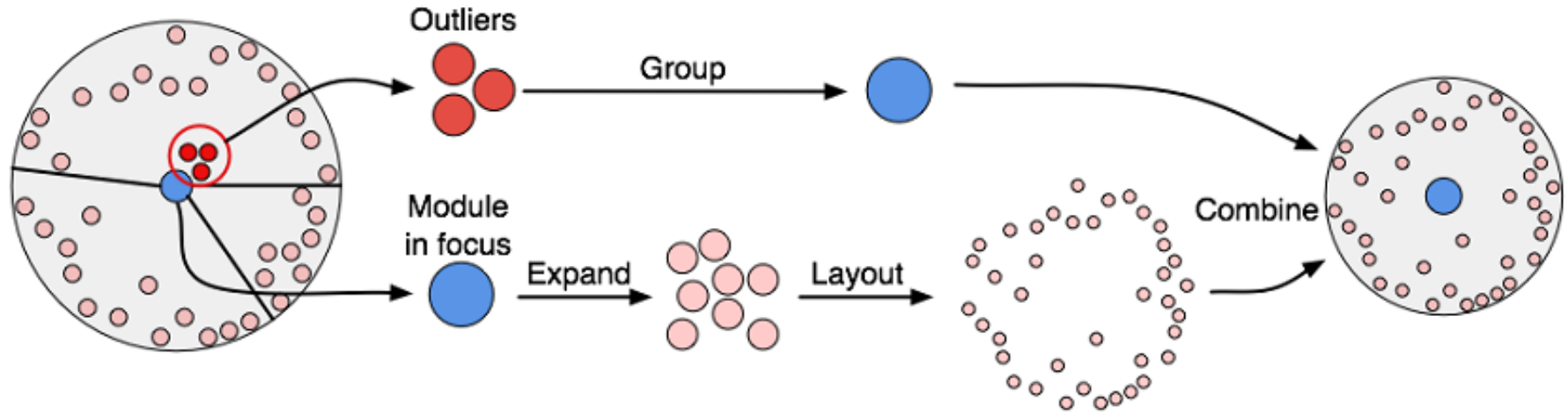


Poderia ser  
qualquer outra  
métrica



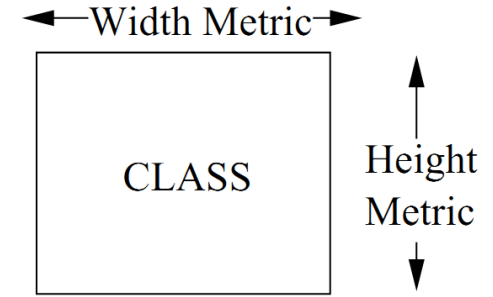
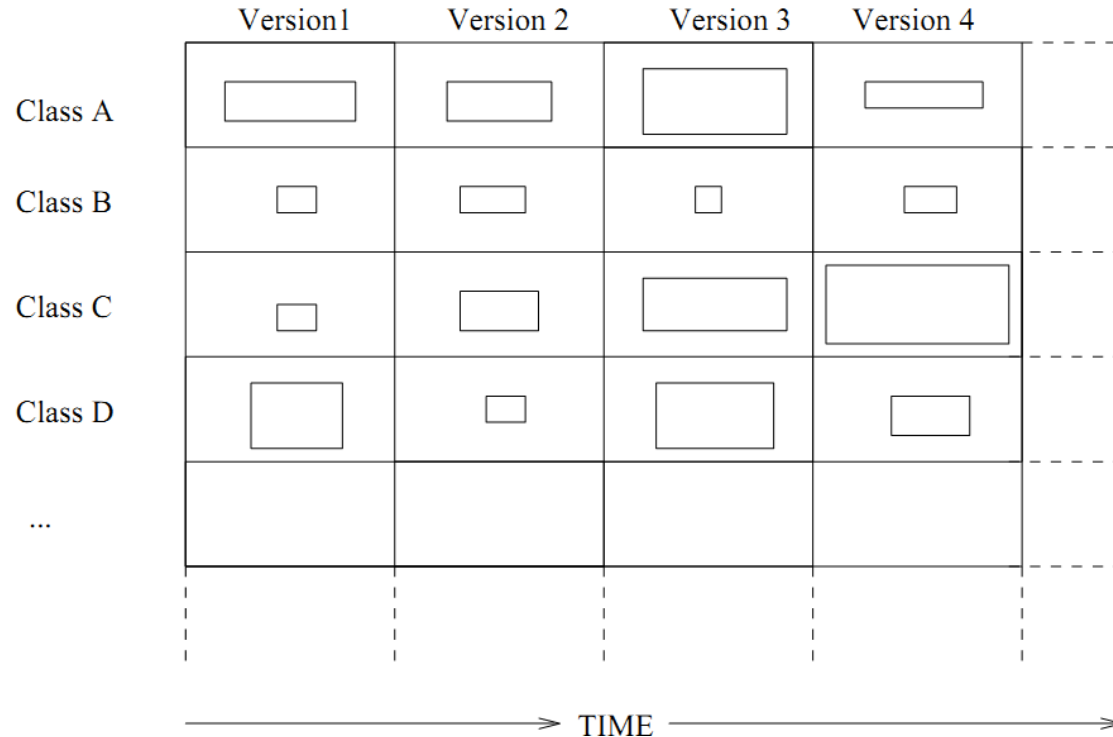
# Visualização de Dependências

73



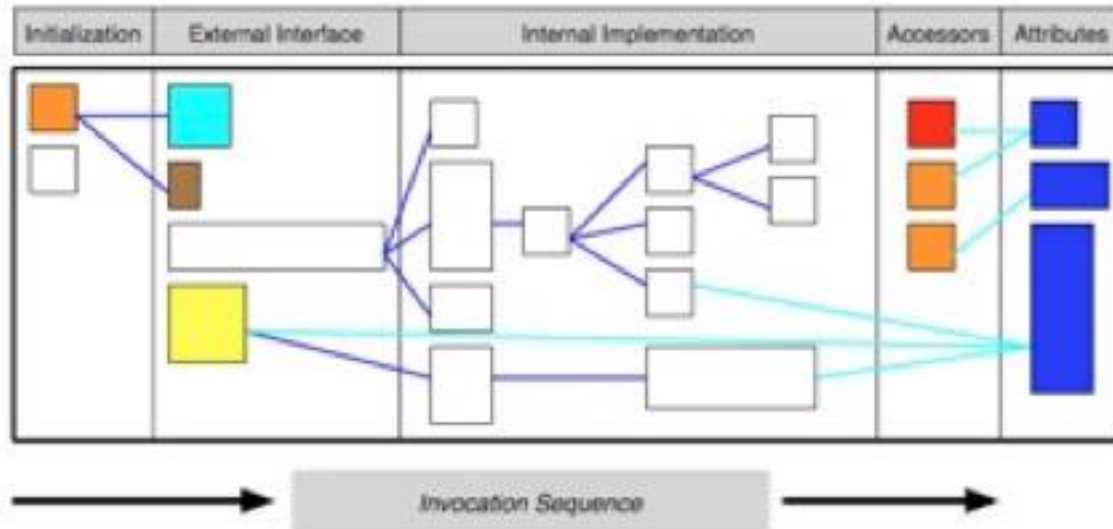
# Evolution Matrix

74



# The Class Blueprint

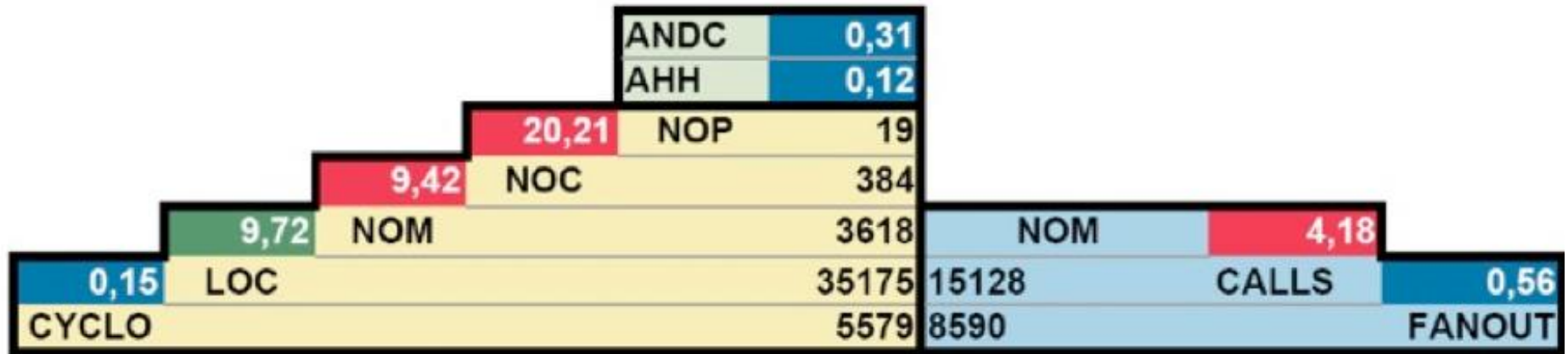
75



# Metrics Pyramid

76

- Diferentes métricas e a relação entre elas
- Cores indicam se valores estão dentro dos limites aceitáveis



# Legenda: Metrics Pyramid

77

- ❑ NOP: Número de pacotes
- ❑ NOC: Número de classes
- ❑ NOM: Número de métodos
- ❑ LOC: Linhas de Código
- ❑ CYCLO: Complexidade ciclomática
- ❑ CALLS: Número de invocações de métodos distintos
- ❑ FANOUT: Número de tipos “de fora” referenciados
- ❑ ANDC: Número médio de classes que foram derivadas
- ❑ AHH: Número médio da árvore de hierarquia

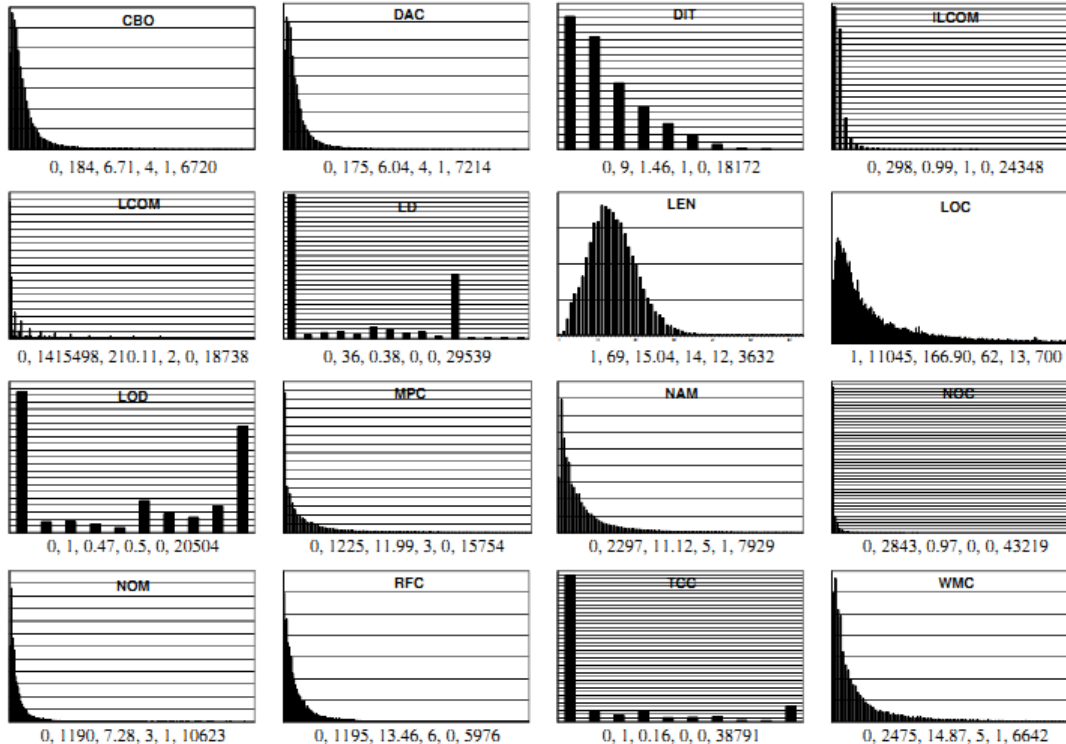
# Métricas de referência

78

|                    | Java |         |      | C++  |         |      |
|--------------------|------|---------|------|------|---------|------|
| Metric             | Low  | Average | High | Low  | Average | High |
| CYCLO/Line of code | 0.16 | 0.20    | 0.24 | 0.20 | 0.25    | 0.30 |
| LOC/Operation      | 7    | 10      | 13   | 5    | 10      | 16   |
| NOM/Class          | 4    | 7       | 10   | 4    | 9       | 15   |
| NOC /Package       | 6    | 17      | 26   | 3    | 19      | 35   |
| CALLS/Operation    | 2.01 | 2.62    | 3.2  | 1.17 | 1.58    | 2    |
| FANOUT /Call       | 0.56 | 0.62    | 0.68 | 0.20 | 0.34    | 0.48 |
| ANDC               | 0.25 | 0.41    | 0.57 | 0.19 | 0.28    | 0.37 |
| AHH                | 0.09 | 0.21    | 0.32 | 0.05 | 0.13    | 0.21 |

# Existem várias!

79



*Quantitative Evaluation of  
Software Quality Metrics in  
Open-Source Projects  
(Barkmann et al.)*

Figure 2. Histograms – statistical metrics values; min, max, average, median, modus, max number.

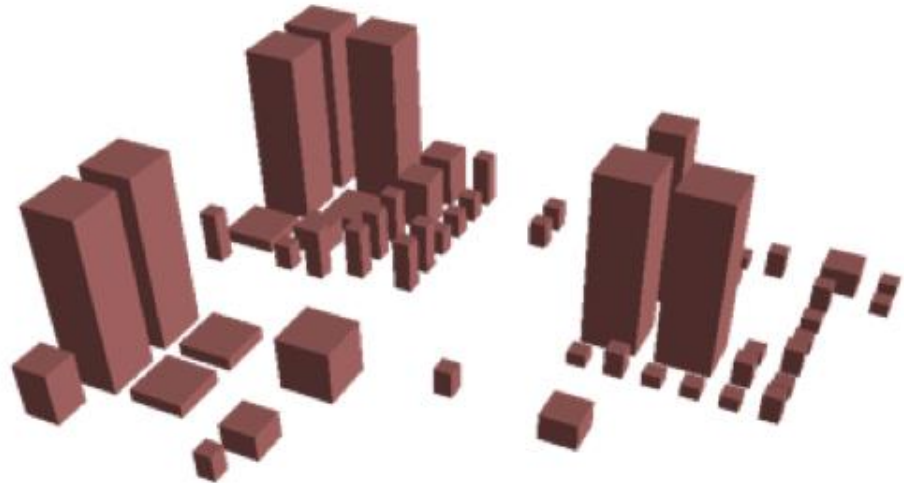
## 80



# Evospaces/Codecity

81

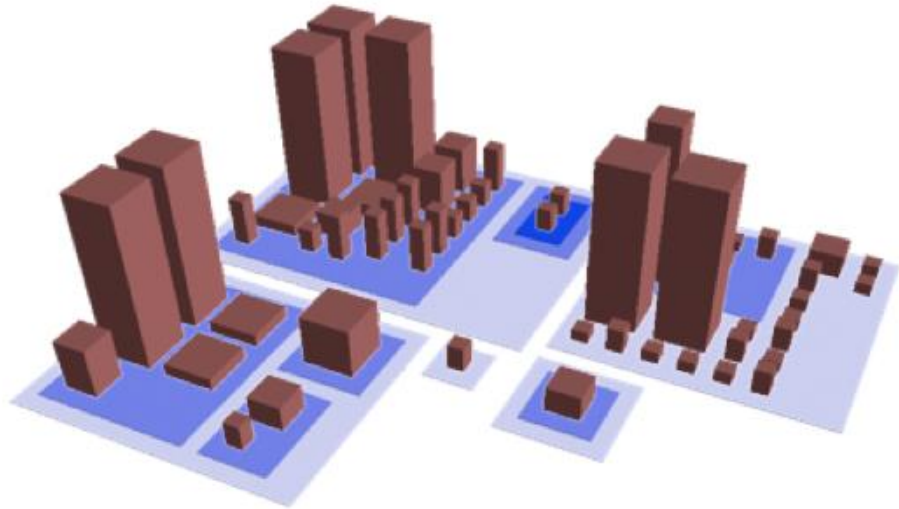
| software | representation |
|----------|----------------|
| classes  | buildings      |
| packages |                |
| system   |                |



# Evospaces/Codecity

82

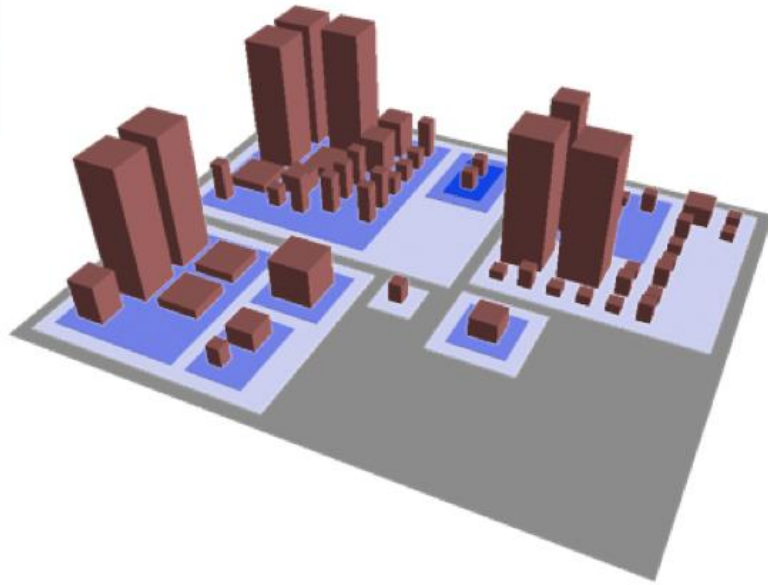
| software | representation |
|----------|----------------|
| classes  | buildings      |
| packages | districts      |
| system   |                |



# Evospaces/Codecity

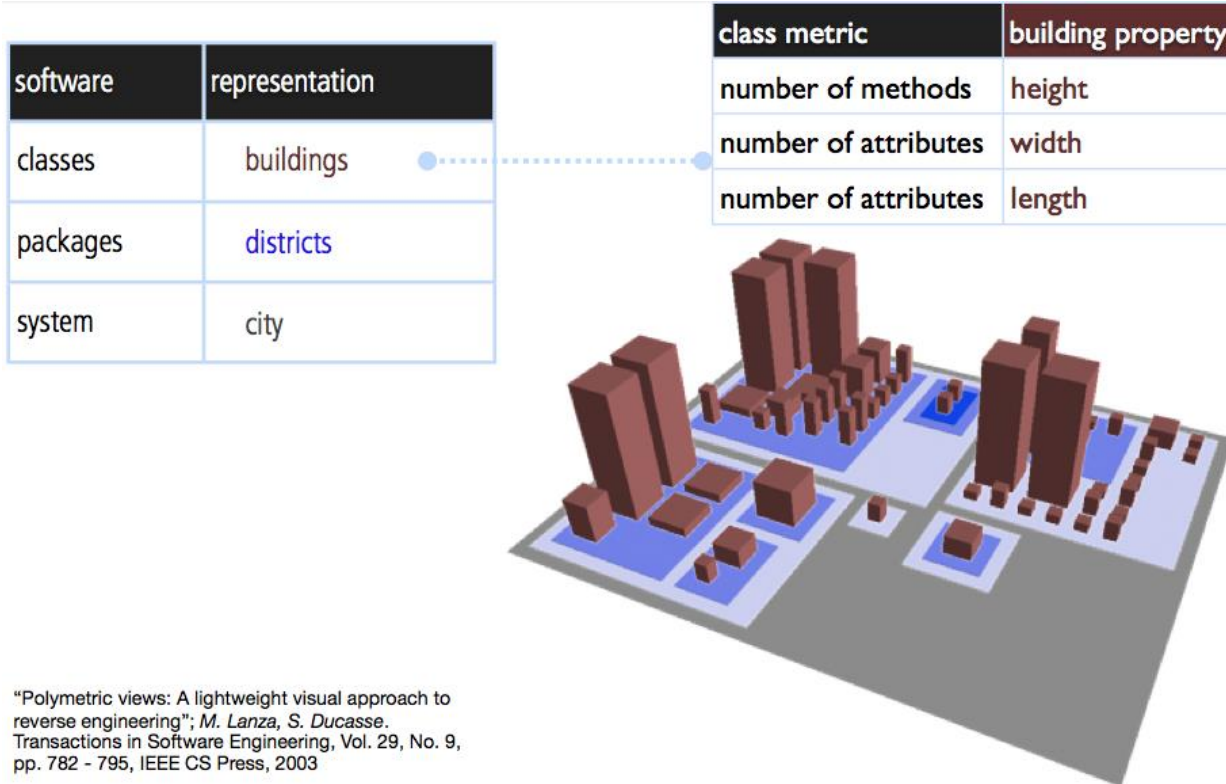
83

| software | representation |
|----------|----------------|
| classes  | buildings      |
| packages | districts      |
| system   | city           |



# Evospaces/Codecity

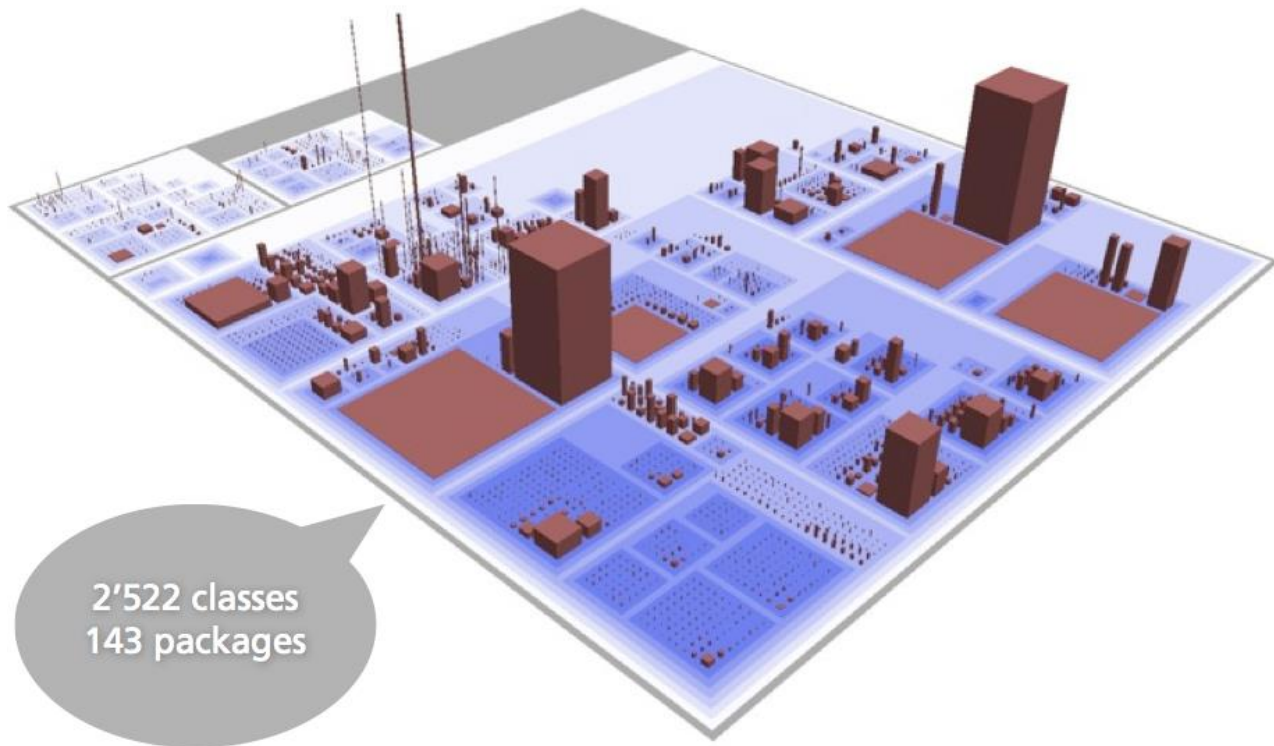
84



"Polymetric views: A lightweight visual approach to reverse engineering"; *M. Lanza, S. Ducasse*. Transactions in Software Engineering, Vol. 29, No. 9, pp. 782 - 795, IEEE CS Press, 2003

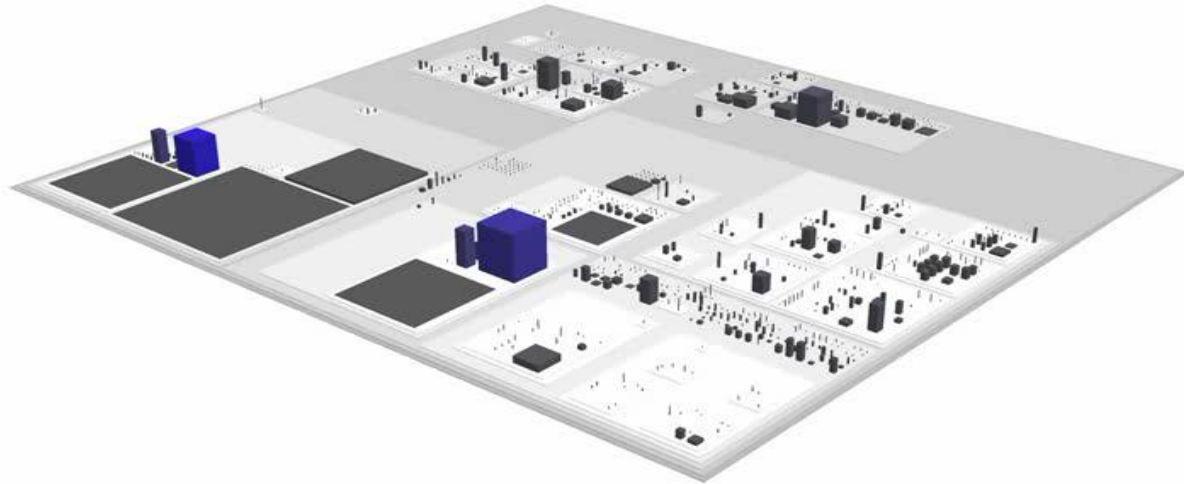
# ArgoUML

85



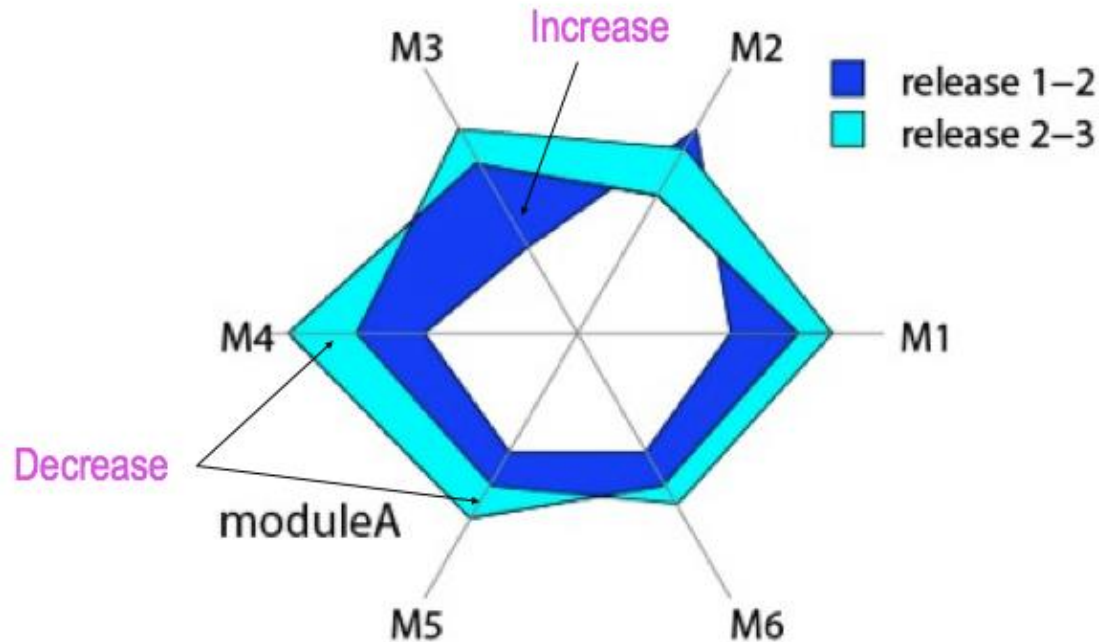
# CodeCity ao longo do tempo

86



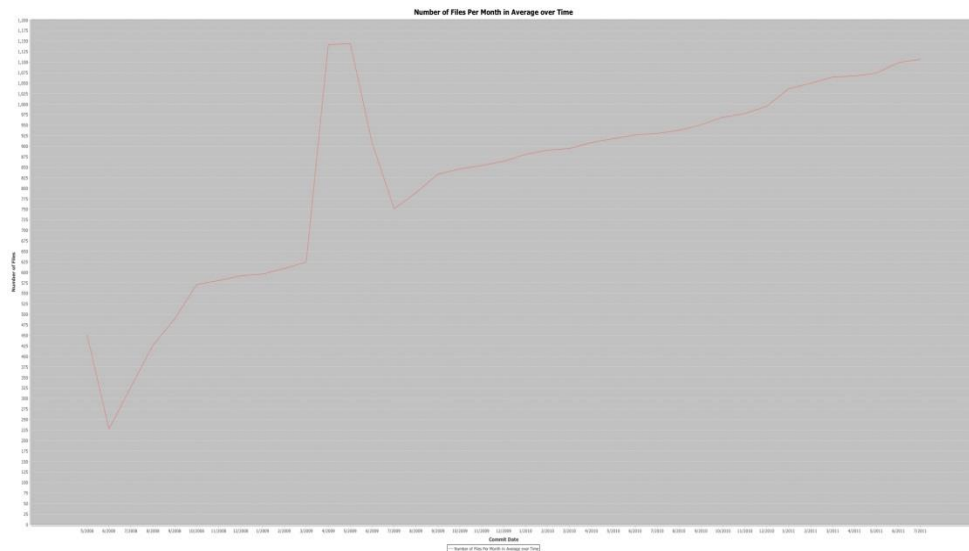
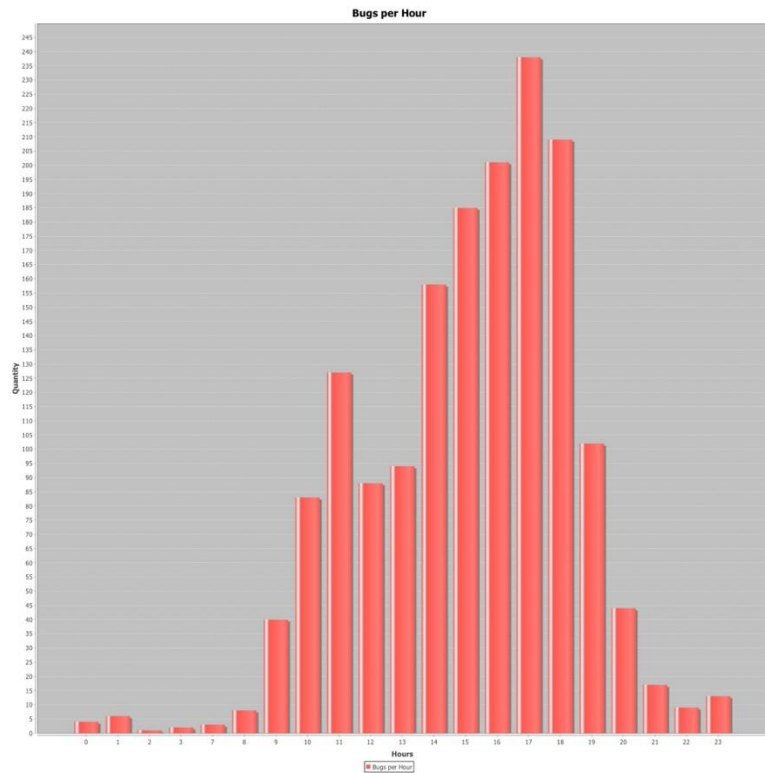
# Diagramas de Kiviat (radares)

87



# Gráficos de barras e linhas

88



# Agenda

89

- Motivação
- Conceitos Básicos
- Temas Atuais de Pesquisa
- Métricas e Visualização
- XFlow and rEvolution

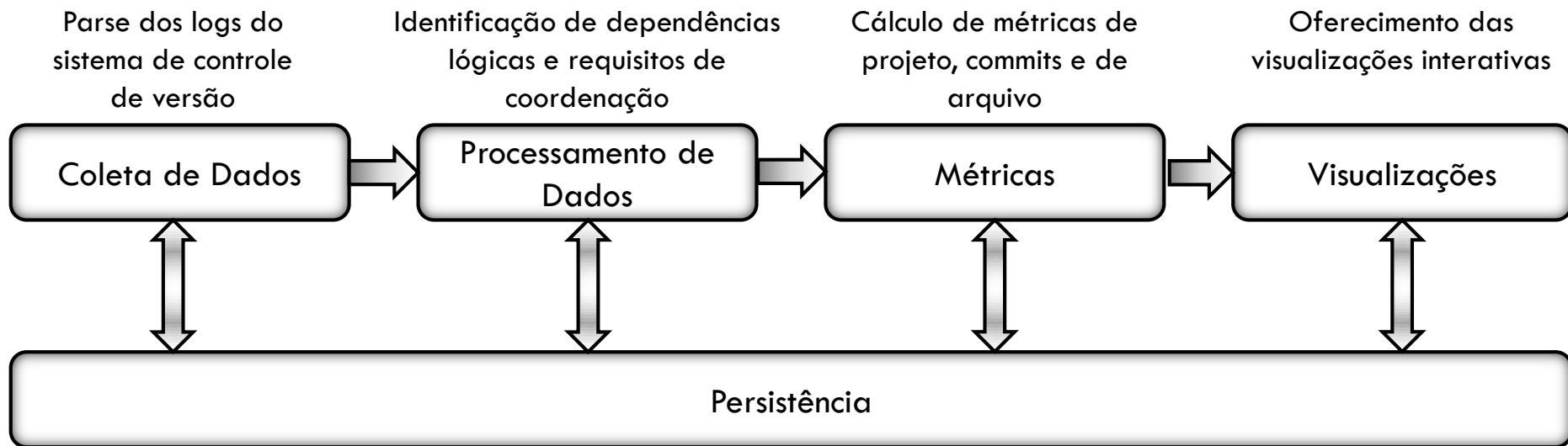
# XFlow

90

- XFlow é uma ferramenta **extensível** que dá suporte a **análises empíricas** em evolução de software
  - ▣ Open-source (GPL)
- Baseada no protótipo TransFlow [Costa *et al.* 2009]
- Apoia a **análise integrada** de **aspectos técnicos e sociais** por meio de ricas visualizações e cálculo de métricas
- Trata problemas de desempenho encontrados em trabalhos relacionados

# Princípios de Funcionamento

91



# Design Rationale

92

- ❑ Foco no apoio à pesquisa empírica
- ❑ Extensibilidade como chave para alcançar completude
- ❑ Faça apenas uma vez
- ❑ Filtre e personalize

# Visualizações da XFlow

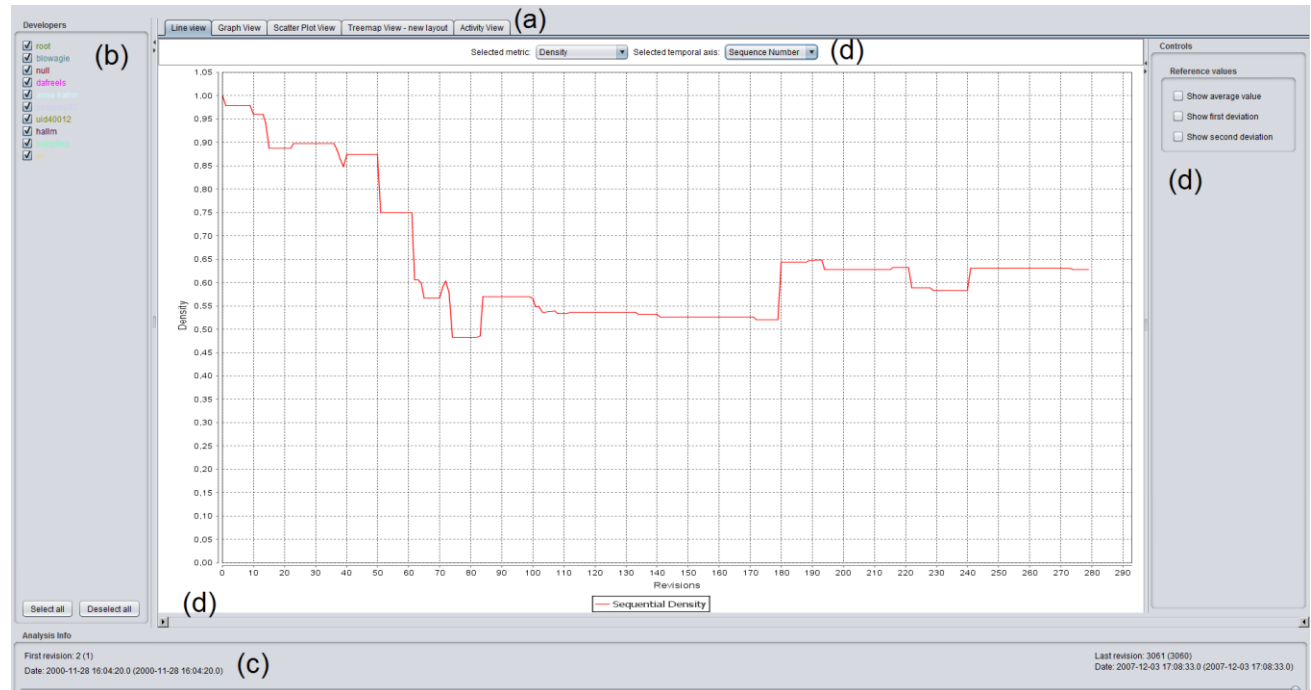
93

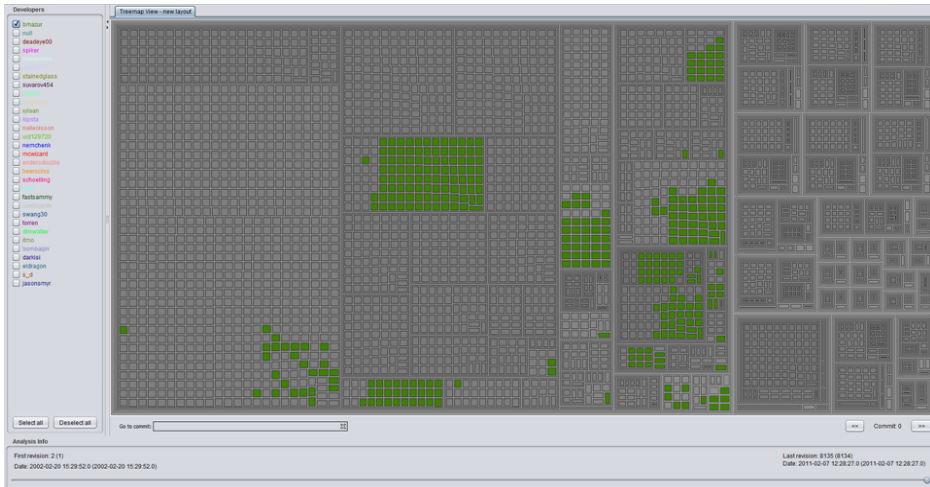
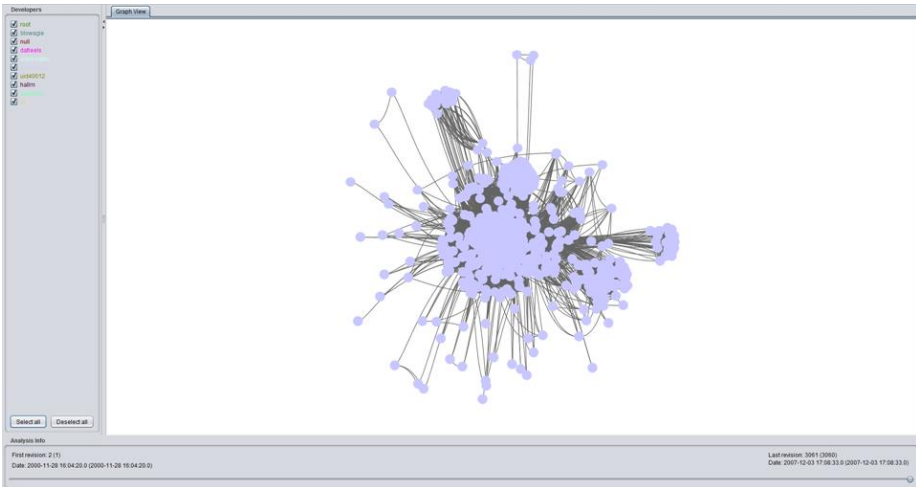
- “Overview primeiro, zoom e filtragem, e então detalhes sob demanda” [Shneiderman 1996]
  - ▣ Alcançada por meio de **filtros e controles**
- Cinco visualizações **interativas**
  - ▣ Gráfico de Linhas
  - ▣ Scatterplot
  - ▣ TreeMap
  - ▣ Grafo
  - ▣ Atividade

# Mecanismos de Interação

94

- (a) Abas para visualizações
- (b) Painel de desenvolvedores
- (c) Barra de informações da análise
- (d) Controles específicos





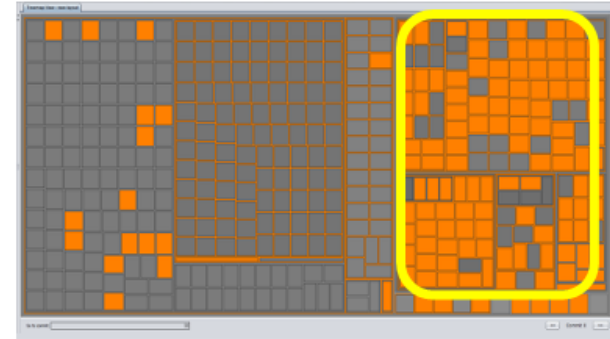
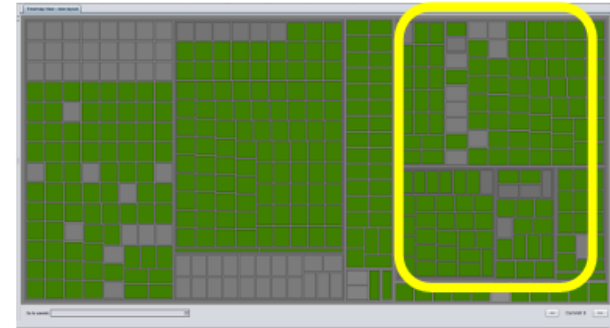
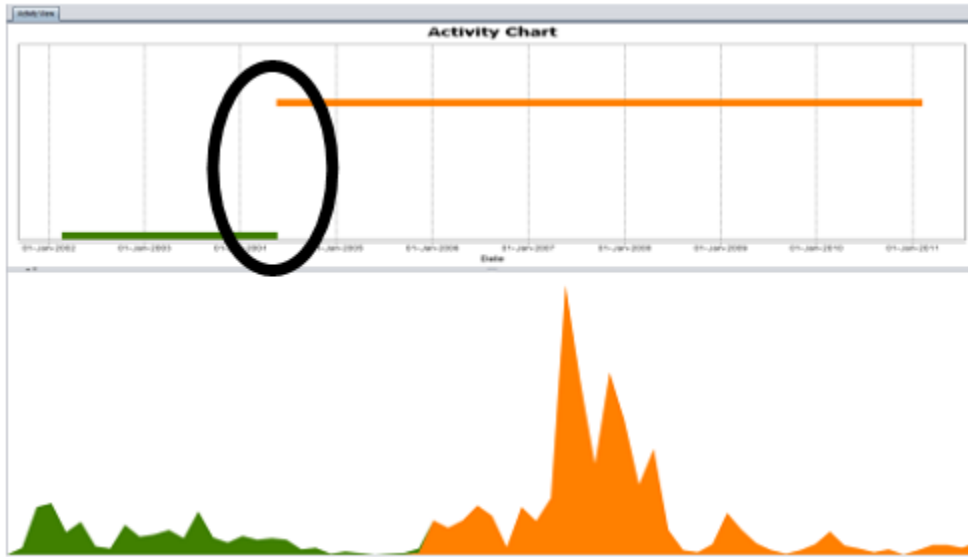
# Cenário Ilustrativo #1

96

- Evolução Sociotécnica de Software
  - ▣ Como *times* de desenvolvimento lidam com a **saída** de desenvolvedores *princípios*?
  - ▣ A **saída** de líderes **comprometeu** o projeto GIMP inteiro (hiato de 20 meses no desenvolvimento) [Ye & Kishida 2003]
  - ▣ Visualizações de **Atividade** e **TreeMap** foram empregadas para analisar o Megamek (BattleTech board game)

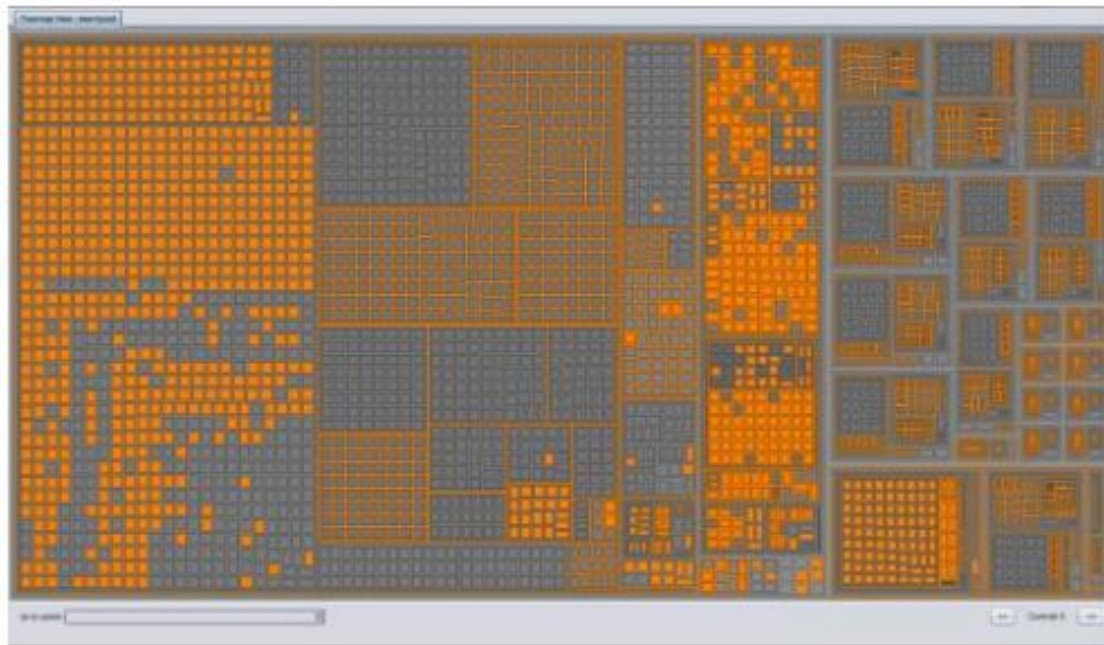
# Cenário Ilustrativo #1

97



# Cenário Ilustrativo #1

98



# Cenário Ilustrativo #2

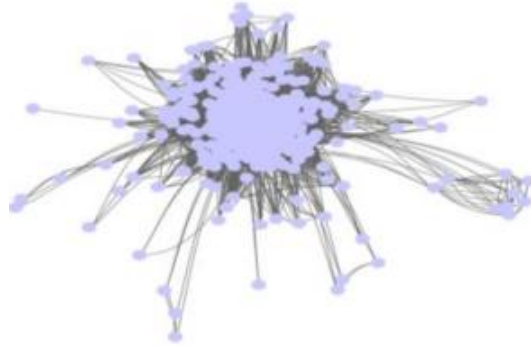
99

- Efeitos da arquitetura sobre desenvolvedores
  - ▣ Como *um sistema de software cresce para **acomodar novas funcionalidades***?
  - ▣ A análise integrada de **dependências lógicas** (foco técnico) e **requisitos de coordenação** (foco social) dá suporte pra este tipo de análise
  - ▣ A visualização de **grafo** foi empregada para analisar ambos Apache Lucene e jEdit (IDE)

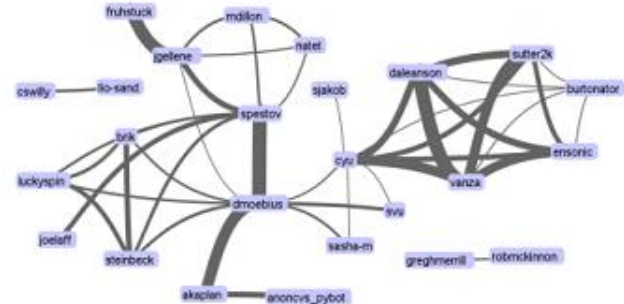
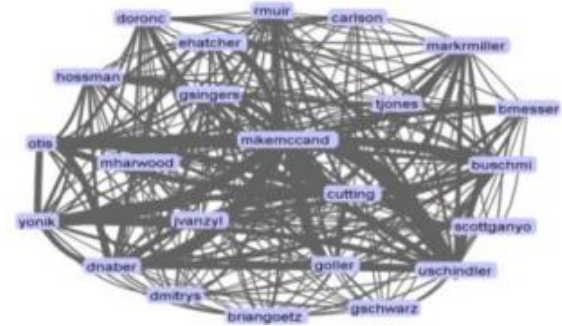
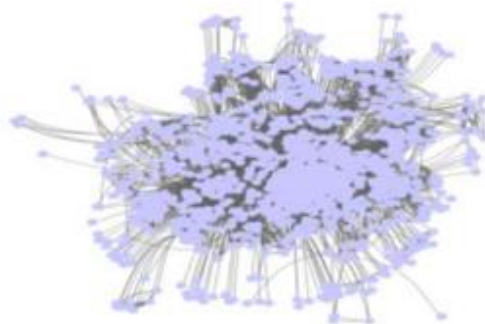
# Cenário Ilustrativo #2

100

Apache Lucene  
Dependency Graphs



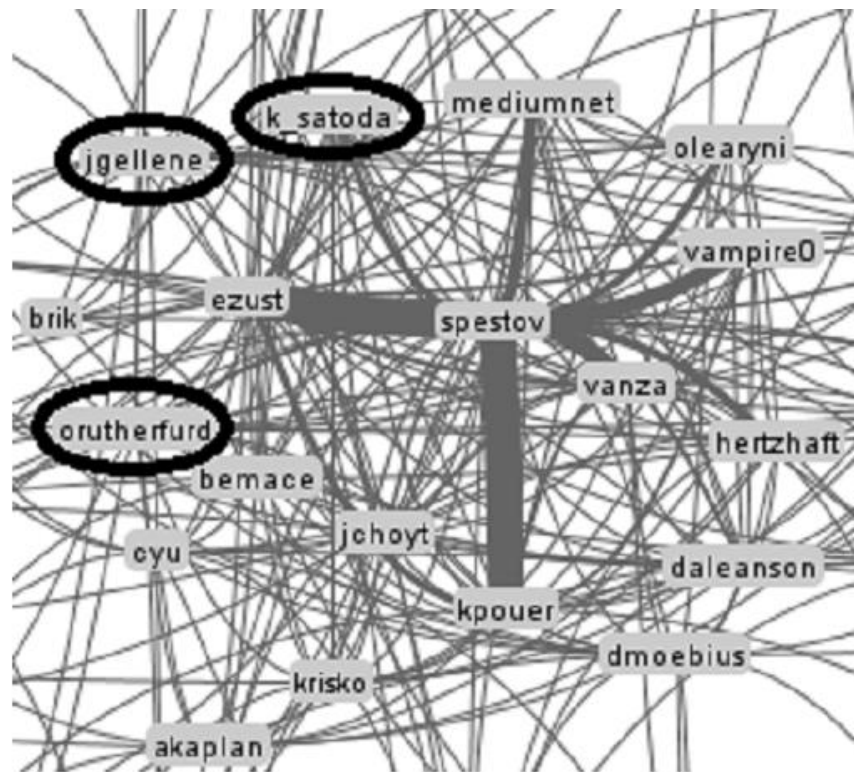
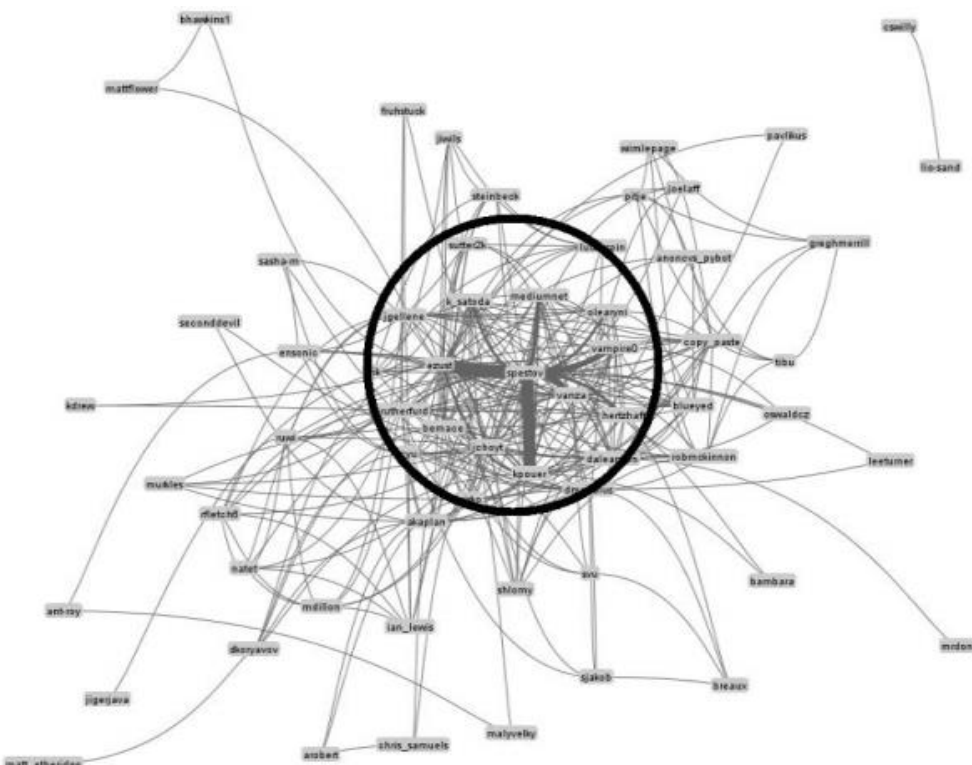
jEdit  
Dependency Graphs



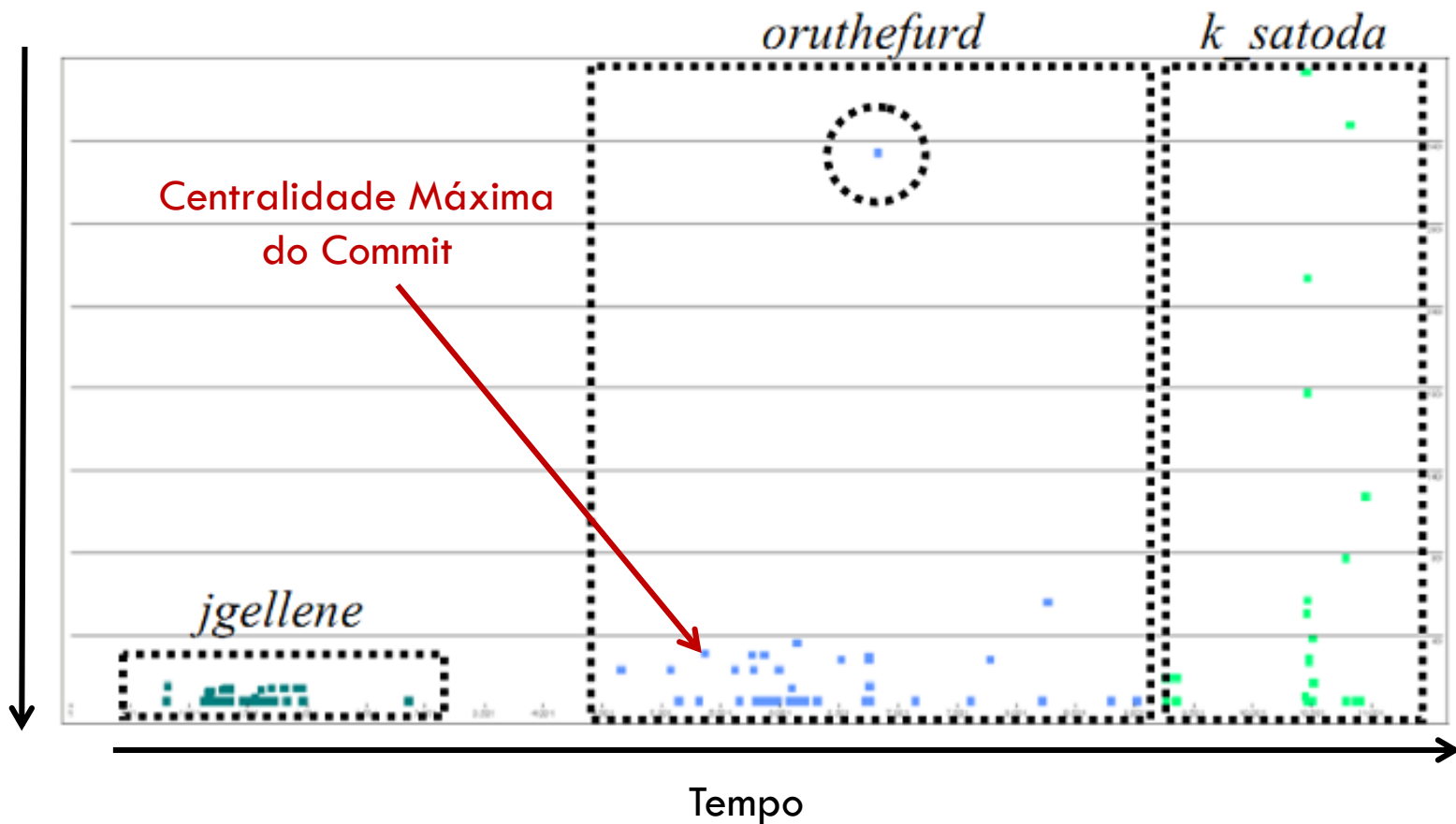
# Cenário Ilustrativo #3

101

- ❑ Compreendendo papel de desenvolvedores
- ❑ Estudo exploratório do projeto jEdit
  - ▣ Coleta: 10895 commits
    - 6 anos e meio de desenvolvimento
  - ▣ Apenas arquivos com extensão Java
- ❑ Análise da contribuição de 3 desenvolvedores
  - ▣ k\_satoda, jgellene e orutherford



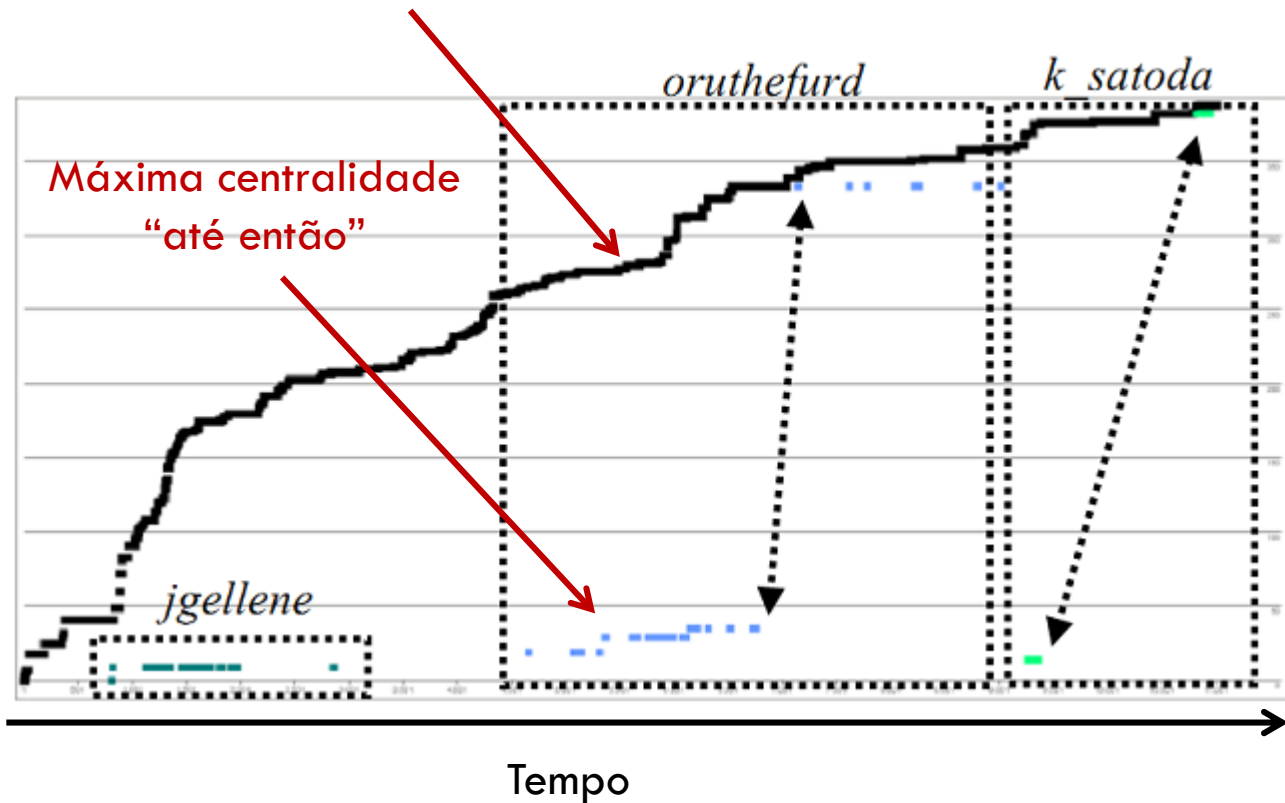
Centralidade

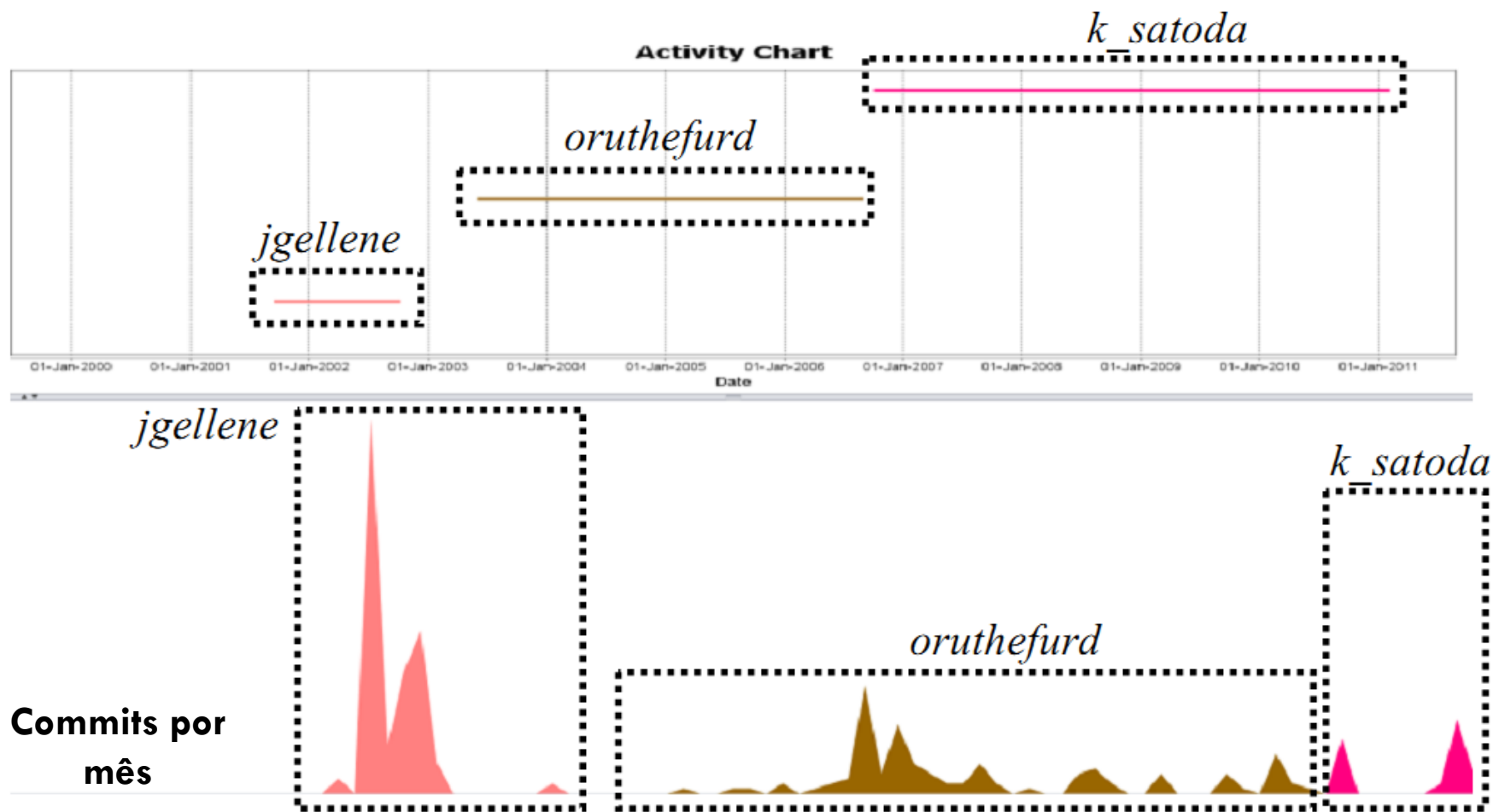


Maior centralidade atingida até o momento  
(valor de referência)

Máxima centralidade  
“até então”

Centralidade





# rEvolution

106

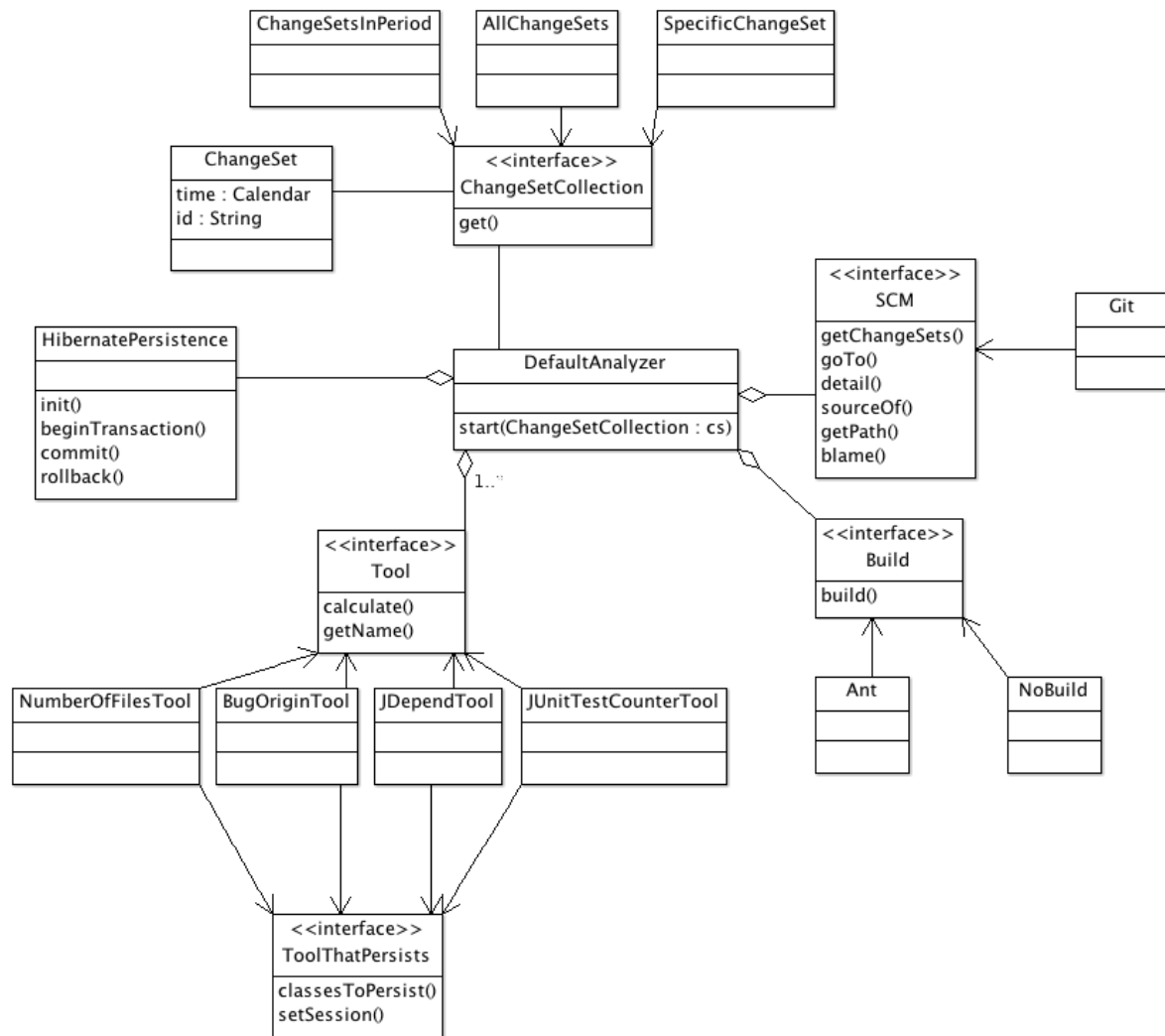
- ❑ Ferramenta que possibilita a geração de gráficos sobre a evolução do software
  - ▣ Analisa informações do repositório de código, como *commit messages*, datas, desenvolvedores, etc.
  - ▣ Executa ferramentas de métricas como JavaNCSS, JDepend, e triangula esses valores
  - ▣ Diversas visualizações já implementadas
- ❑ Ferramenta Open Source que pode ser encontrada em <http://www.github.com/mauricioaniche/rEvolution>
- ❑ Criada por Mauricio Aniche

# rEvolution

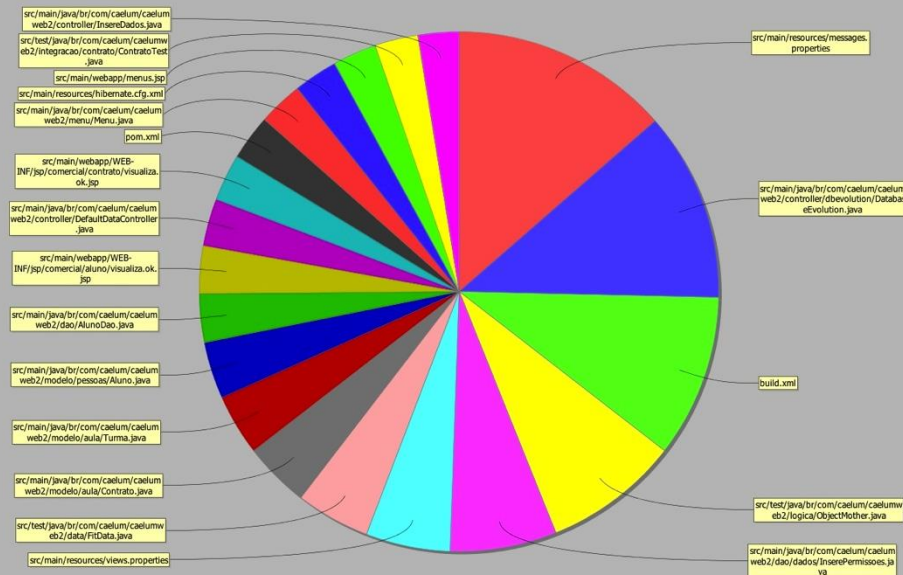
107

- ❑ Atualmente só suporta Git
  - ▣ Implementação de SVN está em andamento
- ❑ Permite a criação de novas visualizações ou medições de maneira fácil
- ❑ Persiste os dados em MySQL
  - ▣ Por usar Hibernate, tira do usuário a necessidade de escrever INSERTs, CREATE TABLEs ou coisa do tipo, ao encaixar uma nova métrica

# Modelo de Classes do rEvolution

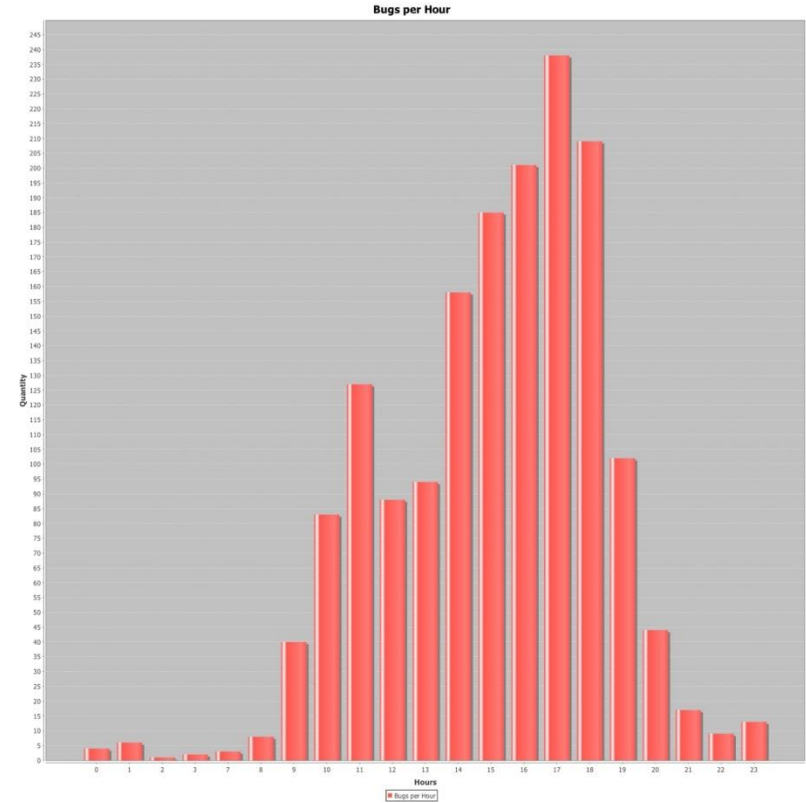
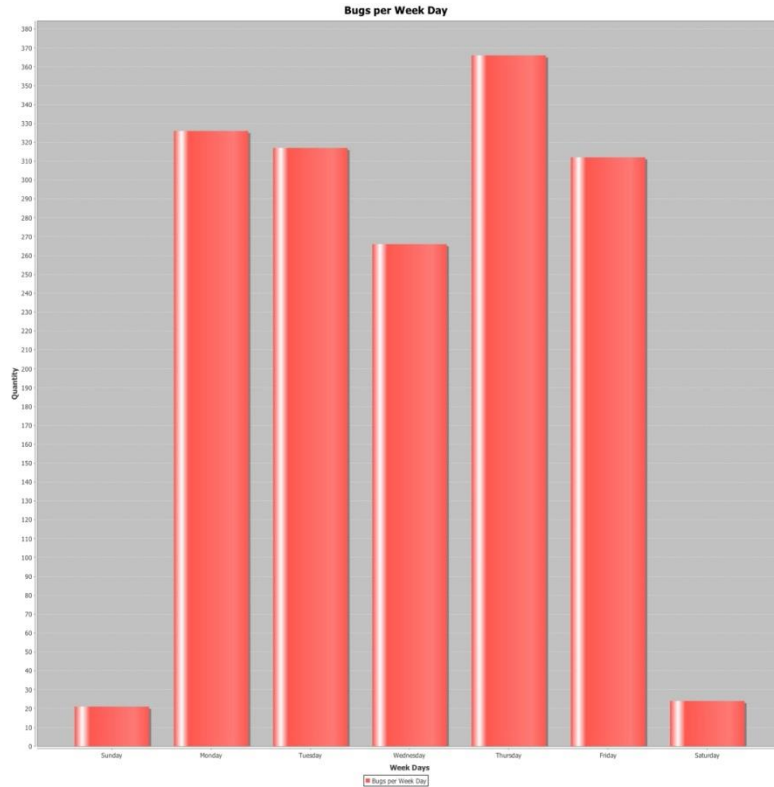


# Artefatos mais modificados

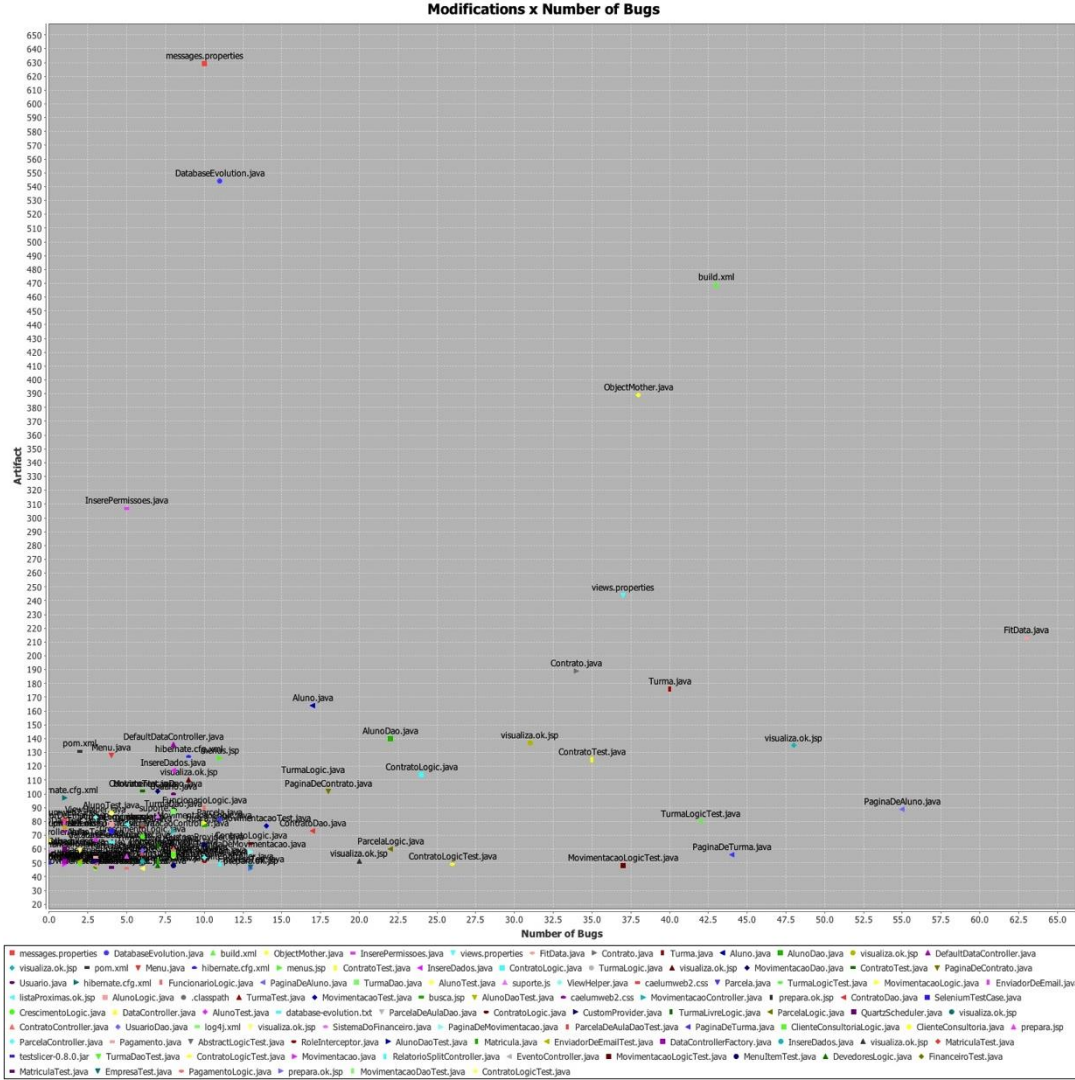


# Bugs por Dia e Hora

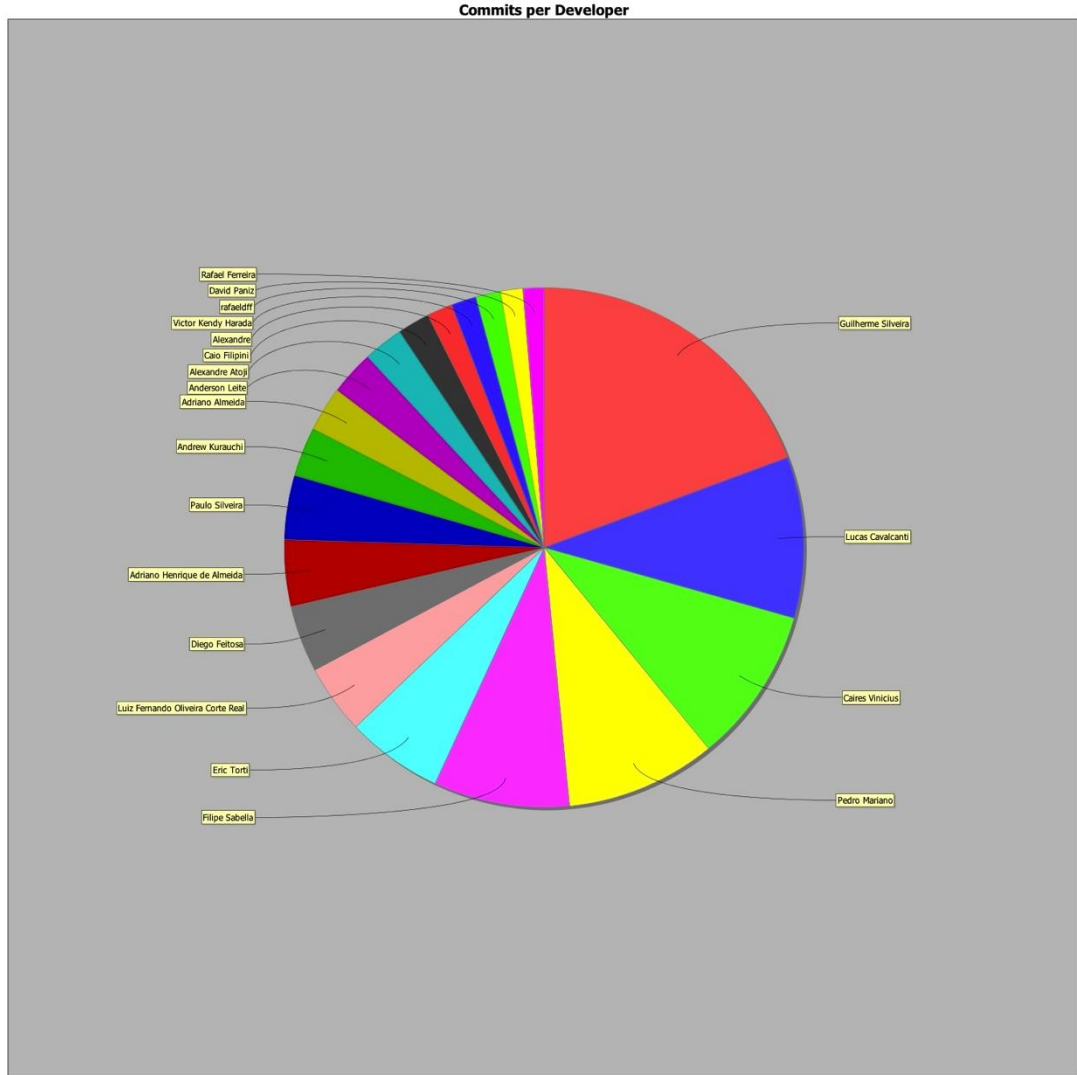
110



## 111

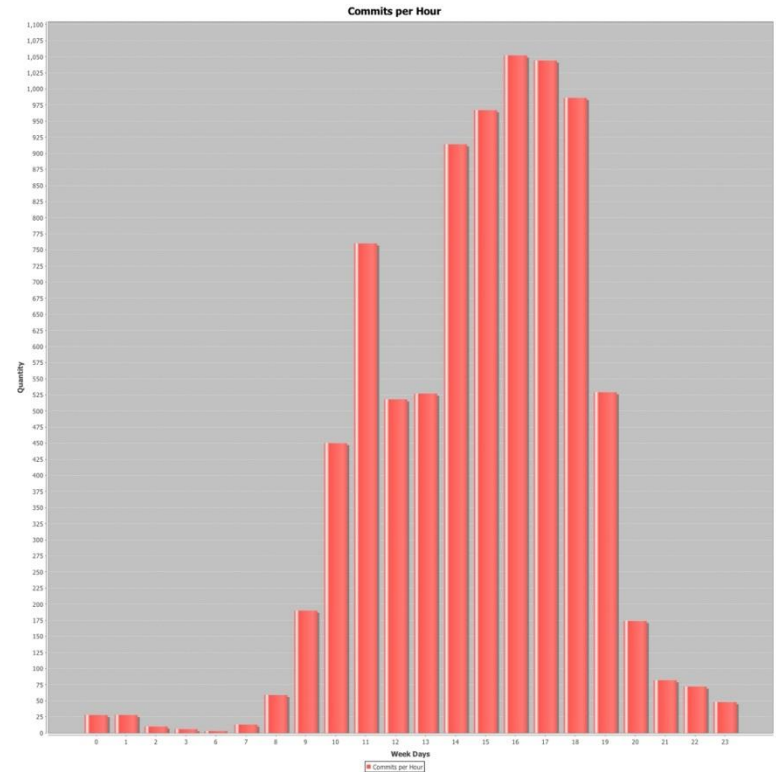
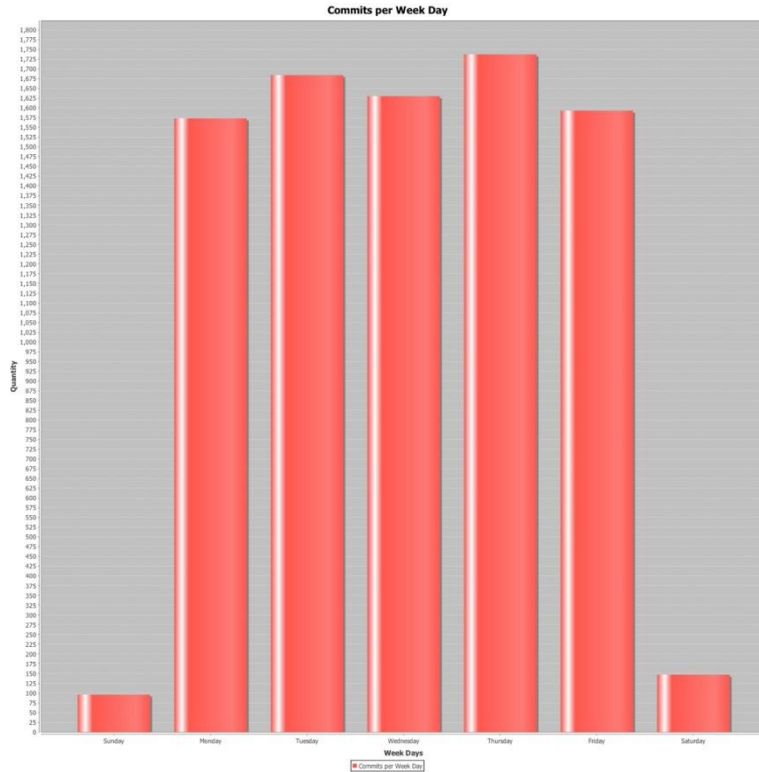


# Commits por desenvolvedor



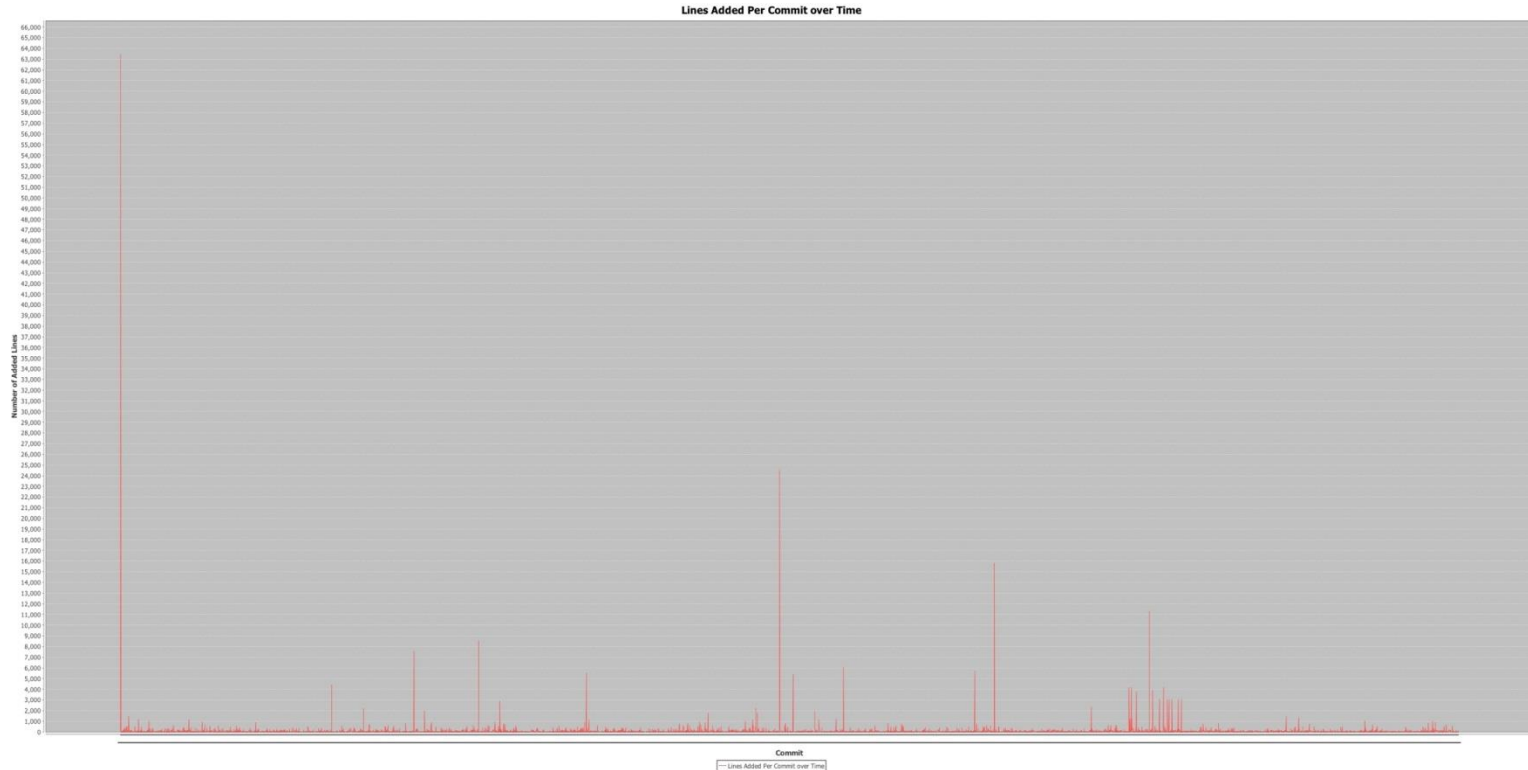
# Commits por Dia e Hora

113



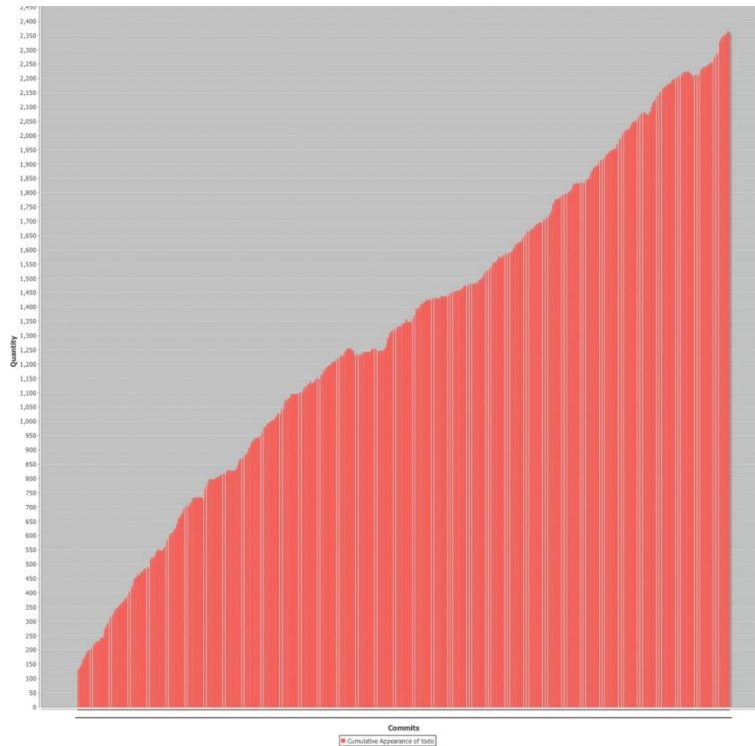
# Linhas adicionadas ao longo do tempo

114



# Testes acumulados ao longo do tempo

115



# Futuro do rEvolution

116

- ❑ Product Backlog já grande
  - ▣ Ideias vindas inclusive de colegas da Alemanha
- ❑ Integração com SVN e Mercurial
- ❑ Interface visual para facilitar configuração
  - ▣ Hoje o usuário configura através de um .properties
- ❑ Mais e mais visualizações...

# Obrigado!

117

- Mauricio Aniche
  - [aniche@ime.usp.br](mailto:aniche@ime.usp.br) / @mauricioaniche
- Gustavo Oliva
  - [goliva@ime.usp.br](mailto:goliva@ime.usp.br) / @golivax
- Marco Aurélio Gerosa
  - [gerosa@ime.usp.br](mailto:gerosa@ime.usp.br)